Transportation Research Part E 212 (2026) 104885

Contents lists available at ScienceDirect

Transportation Research Part E

journal homepage: www.elsevier.com/locate/tre

A collaborative evolutionary reinforcement learning approach to dynamic pickup and delivery challenges

Xiaoying Yang[a,1], Yiran Li[b,1], Tianxiang Cui[a,*], Ruibin Bai[a]

[a] School of Computer Science, University of Nottingham Ningbo China, 199 Taikang East Road, Ningbo, 315100, Zhejiang, China
[b] Network Security Department, Shanxi Police College, Xibeifanglu 799, 030401, Taiyuan, Shanxi, China

ARTICLE INFO

Keywords:
Transportation
Dynamic pickup and delivery
Evolutionary reinforcement learning
Uncertainty

ABSTRACT

In the contemporary landscape of global logistics, the efficient delivery of commodities at scale has become imperative. Central to this challenge is the Dynamic Pickup and Delivery Problem (DPDP). While DPDP is inherently complex due to stochastic orders and real-time requirements, it becomes particularly formidable in specific supply chain variants constrained by strict loading protocols, such as the Last-In-First-Out (LIFO) requirement found in air cargo and side-loading fleets. Traditional heuristics often struggle to escape local optima under such rigid constraints, while standard Reinforcement Learning (RL) faces difficulties with reward sparsity and high-dimensional action spaces. To address these challenges, we propose Evo-RL, a holistic evolutionary reinforcement learning framework. Unlike disjointed hybrid methods, Evo-RL establishes a synergistic closed loop: the RL agent acts as a strategic planner to capture latent spatiotemporal dependencies and generate feasible initial assignments, while a population-based Genetic Algorithm (GA) serves as an optimization engine to refine sequences and provide dense feedback signals. This collaborative mechanism allows the RL agent to overcome reward sparsity by learning from the diverse evolutionary population. We evaluate our approach on the real-world, large-scale DPDP benchmark from the ICAPS 2021 competition. Empirical results demonstrate that Evo-RL significantly outperforms state-of-the-art heuristics and deep RL baselines in terms of solution quality and convergence speed, proving its efficacy in handling dynamic constraints and large-scale complexities.

1. Introduction

The expansion of e-commerce and on-demand services has spurred an escalating need for streamlined delivery operations. Addressing this necessity, the Dynamic Pickup and Delivery Problem (DPDP) has emerged as a critical focus, aiming to enhance route optimization, scheduling efficiency, and resource management (Peppel et al., 2022). Garnering significant attention from both academia and industry leaders, DPDP's relevance spans across multiple sectors, including e-commerce, on-demand offerings, and supply chain logistics. Recent trends indicate that the exponential growth in last-mile delivery volume poses significant challenges to urban traffic and environmental sustainability (Thomas et al., 2020). Consequently, devising an effective DPDP solution stands as a pivotal strategy for businesses, enabling them to navigate dynamic market landscapes and meet evolving consumer expectations effectively.

* Corresponding author.
E-mail address: tianxiang.cui@nottingham.edu.cn (T. Cui).
[1] These authors contributed equally to this work.

https://doi.org/10.1016/j.tre.2026.104885
Received 3 November 2025; Received in revised form 6 April 2026; Accepted 13 April 2026
Available online 7 May 2026
1366-5545/© 2026 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY-NC license (http://creativecommons.org/licenses/by-nc/4.0/).

The concept of DPDP is deeply rooted in the mid-20th century, with its precursor, the Vehicle Routing Problem (VRP), drawing considerable research focus in the 1950s. The VRP was extensively applied in the fields of transportation, logistics, and distribution across a diverse array of industries. It wasn't until the late 1990s that the Pickup and Delivery Problem (PDP) was introduced as a specific offshoot of VRP, tailored to enhance the efficiency of pickup and delivery operations for goods or passengers (Ge et al., 2025). Unlike its dynamic successor, the PDP was set in a static environment where goods were assigned to be transported from their origins to their destinations by trucks, without the inclusion of new, unforeseen orders (Yang et al., 2025). Over time, the PDP has captivated the research community's interest, evolving into one of the most thoroughly investigated combinatorial optimization challenges. It has laid the groundwork for the development of DPDP, enriching the theoretical underpinnings of dynamic routing and scheduling (Hao et al., 2022). The applications of static PDP have become widespread, finding use in areas such as robotics, transportation systems, and logistics, proving its versatility and enduring relevance within the scientific and industrial spheres (Osaba et al., 2019).

The primary goal of PDP centers on optimizing the use of vehicles and minimizing the distances traveled, all while fulfilling the demands of clients at various locations. The complexity of PDP is underscored by its classification as an NP-hard problem, highlighting the inherent challenges in finding optimal solutions. Historically, the bulk of research in the field has concentrated on static PDPs, with numerous studies delving into these scenarios (Liu et al., 2021). In contrast, the dynamic aspects of PDP have been less explored, with a relatively smaller pool of research addressing these more fluid, real-time challenges (Cai et al., 2023). Within the specific context of the air cargo industry or rigidly structured logistics fleets, a common challenge is a static PDP variant that focuses on the transportation of standard unit load devices from freight forwarders' warehouses to the facilities of ground handlers. This particular model is subject to a set of constraints including, but not limited to, capacity limitations, specific time windows, docking considerations, and the Last-In-First-Out (LIFO) loading strategy (Bombelli and Fazi, 2022). The LIFO constraint, in particular, is physically necessitated by the use of side-loading vehicles and narrow storage compartments, where earlier loaded goods remain inaccessible until later items are unloaded. This physical restriction significantly complicates the routing logic, as the simple insertion of a new pickup may violate the unloading sequence of existing cargo. However, such static models may fall short in mirroring the complexities of actual industry conditions, where pickup and delivery demands can vary unpredictably, and uncertainty is a constant factor. To bridge this gap, DPDP has been introduced. DPDP offers a robust framework capable of adapting to these ever-changing scenarios, thereby granting businesses the agility and flexibility needed to efficiently navigate the dynamic landscape of delivery and logistics.

DPDP involves determining an optimal sequence of pickups and deliveries for a set of tasks and a fleet of vehicles, considering dynamic aspects such as changes in the availability of tasks and the status of vehicles over time. In practice, businesses utilizing DPDP must manage a fleet that is not only capable of handling orders generated on the fly but also of transporting them from their origin points to the intended destinations in a timely fashion. During the execution of deliveries, companies face the critical task of selecting the right vehicles for the transportation of goods, ensuring that each item is moved efficiently from the pickup to the delivery point. The introduction of new orders necessitates a recalibration of the existing delivery routes and schedules, a process that must seamlessly integrate with ongoing operations while honouring constraints such as the LIFO principle. DPDP has been widely employed in various applications, including long-distance transportation, multimodal transportation, and less-than-truckload transportation. DPDP is particularly pertinent to some of the world's largest manufacturers, such as Amazon and Huawei, which rely on hundreds of factories to distribute billions of products annually. Given the high volume of delivery requests, even a small improvement in logistics efficiency can result in significant benefits. Therefore, it is crucial to develop an effective optimization method for dispatching orders and planning truck routes. The effective resolution of DPDP can lead to substantial improvements in operational efficiency, cost reduction, and customer satisfaction for these companies and also many others in the logistics and distribution sectors.

Exact methods based on conventional optimization techniques can be used to solve small DPDP instances (Savelsbergh and Sol, 1998). The idea is to divide the DPDP into a number of static problems and optimize each of them using linear or mixed-integer programming. This limitation of exact methods becomes particularly apparent when dealing with large-scale DPDP instances. Since the number of possible combinations grows exponentially with the number of tasks and vehicles, it is impractical to use exact methods for the high-volume order environments that are typical for major logistics operations. Heuristic and metaheuristic methods such as Tabu Search (TS) (Gendreau et al., 2006), Genetic Algorithm (GA) (Sáez et al., 2008), and Evolutionary Algorithm (EA) (Muñoz-Carpintero et al., 2015), can also be used to find good quality solutions for DPDP instances within a reasonable amount of time, but once the problem conditions change slightly, they need to be revised (Cai et al., 2024).These algorithms are notably effective as black-box optimizers, i.e., for solving issues with difficult mathematical formalization. However, they do not take advantage of potential dynamic feedback signals available in the environment, which may limit their ability to find optimal solutions in dynamic and complex problem domains. Due to the unpredictable nature of manufacturing processes and requirements, the majority of delivery requirements in the DPDP cannot be forecasted in advance. Meanwhile, the dynamic changes require real-time decision-making and the ability to quickly adapt to the assignment and scheduling of the tasks. As a result, it is difficult to deploy heuristic and metaheuristic methods in practice due to the high level of uncertainties.

Recently, Reinforcement Learning (RL) algorithms have been proven effective in sequential decision-making problems, especially in the presence of uncertainties (Zhang et al., 2022; Cui et al., 2023, 2024; Jin et al., 2023). In particular, RL has the potential to enhance the "myopic vision" of heuristic and metaheuristic algorithms (Kool et al., 2019). However, vanilla RL approaches encounter inherent challenges when applied to dynamic industrial problems, such as DPDP. These challenges primarily stem from the exploration-exploitation trade-off, constraint handling, and reward sparsity (Dulac-Arnold et al., 2021; Fruit et al., 2018). While hybrid methods combining RL with local search strategies like Adaptive Large Neighborhood Search (ALNS) are well-established, they typically rely on single-trajectory search mechanisms. In contrast, the population-based evolutionary approach offers distinct advantages in this context. Unlike ALNS, which iteratively refines a single solution, the evolutionary approach like GA maintains a

diverse solution population. This diversity generates a rich stream of exploration data for the RL agent, effectively mitigating reward sparsity and preventing policy collapse within the high-dimensional action space of DPDP.

In the ICAPS 2021 competition, Huawei raised the level of realism and complexity in the dynamic DPDP challenge (Hao et al., 2022). The benchmark focuses on a practical variation of the one-to-one PDP and includes inputs such as vehicle and order information, constraints including capacity, time windows, dock availability, and the LIFO loading policy, output, and objective functions. In this work, we propose an Evolutionary Reinforcement Learning (Evo-RL) framework applied to the original ICAPS 2021 benchmark. Crucially, we augment the problem formulation (Hao et al., 2022) to explicitly capture the simulator's inherent dynamics. By mathematically modeling stochastic factors, specifically delivery time fluctuations and travel distance variations, we bridge the gap between the theoretical problem and the realistic, stochastic execution environment. The proposed Evo-RL framework addresses these unique challenges by combining the strengths of GA and RL in a synergistic manner. Specifically, Evo-RL decomposes the DPDP into two manageable sub-problems: order allocation (handled by RL) and route planning (optimized by GA). The GA component maintains a population of diverse solutions and uses evolutionary operations to explore the solution space broadly, which helps overcome the exploration limitations and reward sparsity issues typical in conventional RL. Meanwhile, the RL agent utilizes an intrinsic reward function, designed based on our modified problem formulation, to guide learning under dynamic conditions.

Our main contributions are listed as the following:

- We introduce an Evo-RL approach that synergistically integrates a GA with conventional RL techniques. The proposed method employs the GA to enhance and refine the solutions initially produced by RL through iterative process, while concurrently ensuring that the evolved solutions remain compliant with the predefined constraints via evolutionary operations.
- We propose a mathematical model to explicitly account for stochastic delivery times and travel distances, integrating these uncertainties into the standard DPDP formulation. This model underpins a novel intrinsic reward function that guides the RL agent to learn robust policies, ensuring performance stability under real-world perturbations.
- The proposed Evo-RL method undergoes rigorous empirical testing using real-world benchmarks from the 2021 ICAPS competition. The outcomes of this validation process reveal that the proposed Evo-RL method surpasses existing heuristics and traditional DRL methods.

The remaining sections of this paper are organized as follows. In Section 2, we provide a literature review of relevant research. Section 3 describes the problem and presents the modified mathematical model. Section 4 introduces our proposed Evo-RL framework. The experimental results are reported in Section 5. Section 6 concludes the paper.

2. Related work

This section reviews the evolution of the PDP from its static origins to dynamic variants, followed by a detailed analysis of problem constraints and a comprehensive review of methodological approaches.

2.1. Problem variants, constraints, and complexity

The PDP has been one of the most extensively studied network logistic challenges and has received significant attention in recent decades (Cai et al., 2023). While early research focused on static environments, the Dynamic PDP (DPDP) represents a critical branch dealing with real-time operations. DPDPs are often referred to transportation of products from multiple origins to multiple destinations, as described in more detail in Psaraftis et al. (2016). DPDP has practical applications in door-to-door transportation services and courier services. Due to its high complexity, research on DPDPs is less common than that on traditional VRP (Jia et al., 2021; Bai et al., 2023).

To facilitate a better understanding of the table and provide a thorough analysis of the constraints inherent in DPDPs, the following constraints and uncertain factors should be taken into consideration: **Pickup and delivery constraint**: This constraint requires that each item is first picked up and then delivered to its assigned destination by the same vehicle, ensuring goods are not mixed or misplaced during transportation. **Dock availability constraint**: Dock availability at pickup and delivery locations impacts scheduling and operational efficiency, simulating real-world scenarios where the algorithm must arrange docking facilities for prompt loading and unloading. **Vehicles capacity constraint**: This constraint ensures that the total load does not exceed the vehicle's maximum capacity. **Loading cost constraint**: This constraint accounts for the time required to load goods, aiming to minimize loading time, which can vary depending on the size or complexity of the goods. **Time windows constraint**: The time windows constraint specifies that each order's completion time should not exceed the committed maximum service time, though it may be relaxed to a soft constraint in some cases. **Split demand constraint**: When an order's demand exceeds a single vehicle's capacity, the split demand constraint requires dividing the order into smaller units (e.g., one pallet), each served by different vehicles, to accommodate capacity limitations. **LIFO constraint**: In certain scenarios, a LIFO constraint means the most recently picked-up item is delivered first, minimizing loading and unloading times. **Dynamic demand**: Dynamic demand reflects the changing nature of orders due to customer behavior or dispatching variations, requiring adaptive strategies. **Uncertain order**: The uncertain order constraint deals with unknown availability or characteristics of orders at planning or dispatching time, demanding flexibility in resource allocation and scheduling. **Delivery time**: Each order must be delivered within a specified time frame, and the completion time should not exceed the committed maximum service time to avoid penalties or customer dissatisfaction; thus, flexible scheduling and routing strategies are necessary to adapt to real-time changes and optimize delivery time. **Delivery distance**: Finally, minimizing the total delivery distance is crucial for reducing

Table 1
Literature review of DPDP variants.

Paper	Methodology	Constraints						Uncertain factors			
		Dock Avail.	Cap.	Load. Cost	TW	Split	LIFO	Dyn.	Ord.	Time	Dist.
Jeong et al. Jeong and Moon (2024)	Exact (Rolling Horizon)	✗	✓	✓	✓	✗	✗	✓	✗	✗	✗
Swihart et al. Swihart and Papastavrou (1999)	Exact (Stochastic)	✗	✓	✗	✗	✗	✗	✓	✗	✗	✗
Ghiani et al. Ghiani et al. (2009)	Heuristic (Anticipatory)	✗	✗	✗	✗	✗	✗	✓	✗	✗	✗
Pureza et al. Pureza and Laporte (2008)	Heuristic (Waiting)	✗	✓	✓	✓	✗	✗	✓	✗	✗	✗
Mitrovic et al. Mitrović-Minić et al. (2004)	Heuristic (Rolling Horizon)	✗	✗	✓	✓	✗	✗	✓	✗	✗	✗
Gendreau et al. Gendreau et al. (2006)	Heuristic (TS)	✗	✗	✓	✓	✗	✗	✓	✓	✗	✗
Munoz et al. Muñoz-Carpintero et al. (2015)	Heuristic (EA Variant)	✗	✓	✓	✓	✗	✗	✓	✓	✗	✗
Ma et al. Ma et al. (2021)	DRL (Hierarchical)	✓	✓	✓	✓	✓	✓	✓	✗	✗	✗
Du et al. Du et al. (2023)	Heuristic (Hierarchical)	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Li et al. Li et al. (2025)	RL (Hierarchical DRL)	✗	✓	✗	✓	✗	✗	✓	✓	✓	✓
Xu et al. Xu and Wei (2023)	Hybrid (DRL + ALNS)	✗	✓	✗	✓	✗	✓	✓	✓	✓	✓
This Work	**Evo-RL**	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

fuel consumption, transportation costs, and environmental impact, and the algorithm should continuously update routes based on uncertainties to achieve efficient and cost-effective delivery operations. Table 1 summarizes relevant studies on DPDP, comparing them against the constraints listed above. Unlike previous works that address subsets of these challenges, our proposed work aims to tackle the full spectrum of constraints and uncertain factors, demonstrating the comprehensiveness of our problem formulation.

2.2. Methodological approaches

Current methodologies for solving DPDPs can be broadly categorized into Traditional Optimization (Exact and Heuristics), Sequential Decision-Making (ADP and RL), and Hybrid Approaches.

2.2.1. Traditional optimization: Exact and heuristic methods

The first category addresses DPDP by decomposing the dynamic problem into a series of static sub-problems. Savelsbergh and Sol (1998) introduced an independent vehicle dynamic routing algorithm named DRIVE. This algorithm breaks down the target problem into a series of static optimization problems using a subset of current delivery orders and a rolling-horizon framework. To balance solution speed and quality in a dynamic setting, the static optimization problem is tackled using approximation, incomplete optimization methodologies, as well as a sophisticated column management strategy. Swihart and Papastavrou (1999) presented a stochastic and dynamic model for DPDP, where the service demand is generated over time via a Poisson process. Recently, Wolfinger and Salazar-González (2021) developed a branch-and-cut approach for split loads, while Jeong and Moon (2024) applied exact optimization to autonomous delivery robots. However, these approaches exhibit significant computational complexity because each static subproblem must be optimized sequentially.

To further balance solution quality and computational time, meta-heuristic algorithms offer a robust alternative. These approaches can be broadly classified into single-point and population-based methods. Single-point meta-heuristics focus on iteratively improving a single candidate solution using mechanisms like local search, simulated annealing, and TS. For example, Karami et al. (Mitrović-Minić et al., 2004) proposed a rolling-horizon-based method to address same-day pickup and delivery requests by hybridizing heuristics with local search. However, the rolling-horizon approach does not guarantee global optimality over the entire time horizon due to the lack of complete future information. Gendreau et al. (2006) developed a TS heuristic that explores new solutions using a neighbourhood structure built on ejection chains. While effective for local refinement, these methods risk getting trapped in local optima due to limited exploration capabilities. In contrast, population-based meta-heuristics, such as GA and PSO, maintain a diverse set of solutions to explore the search space globally, as comprehensively reviewed by Omidvar et al. (2021). These methods leverage operations like selection, crossover, and mutation to evolve superior solutions over generations. Population-based algorithms have been extensively utilized in VRP contexts (Jia et al., 2021; Xiao et al., 2019; Feng et al., 2020; Duan et al., 2021; Xue et al., 2025; Hu et al., 2025b). For DPDP specifically, recent works combine evolutionary frameworks with local search mechanisms. Muñoz-Carpintero et al. (2015) proposed a variant of EAs where parameters are tuned via multi-objective optimization to trade off performance and computation. Notably, Cai et al. (2024) proposed a decomposition-based multiobjective evolutionary algorithm integrated with TS (MOEA/D-TS) to balance diversity and convergence. However, despite their global search ability, these methods often restart optimization from scratch for each new event, reacting myopically to immediate demands without learning long-term patterns.

2.2.2. Sequential decision-making: From ADP to deep RL

To address the sequential nature of DPDP, researchers model the problem as a Markov Decision Process (MDP). Both Approximate Dynamic Programming (ADP) and Reinforcement Learning (RL) share the fundamental goal of solving these MDPs by approximating future rewards to alleviate the “curse of dimensionality” (Mes and Rivera, 2017). While traditionally distinct communities, they are increasingly viewed as equivalent frameworks where ADP often emphasizes model-based value approximation and RL focuses on data-driven learning.

In the context of logistics, ADP has been extensively applied to handle stochasticity. Recent studies have demonstrated ADP's efficacy in complex scenarios: Agussurja et al. (2019) utilized state aggregation for stochastic ride-sharing; Al-Kanj et al. (2020) applied it to autonomous electric fleet planning; and Mousavi et al. (2024) developed an ADP framework for crowd-shipping with spatiotemporal uncertainty. A theoretical foundation for safe approximation in such dynamic systems was further established by Chakrabarty et al. (2020). Within this framework, two primary approximation strategies are prevalent: Cost Function Approximation (CFA) and Value Function Approximation (VFA). Cost function approximation methods aim to manipulate constraints or objective functions to account for uncertainty implicitly. For instance, safety buffers (Ulmer et al., 2021) can be introduced to pickup times to accommodate potential delays. Scenario-based methodologies also fall under this umbrella, sampling future realizations to optimize current decisions. However, these approaches often struggle with the computational burden of real-time simulation horizons. Value function approximation (VFA) methods, conversely, aim to explicitly estimate the long-term value of current states. As explored in Chen et al. (2022), VFA approximates the value function to guide decision-making without exhaustive lookahead. A critical challenge in VFA is the representation of the vast state space. Traditionally, this relies on domain-specific feature engineering, such as manually aggregating vehicle locations or demand densities. However, designing effective manual features for the highly constrained DPDP with LIFO requires deep domain expertise and limits scalability (Ghiani et al., 2022).

To automate feature extraction and policy learning, Deep Reinforcement Learning (DRL) has emerged as a powerful alternative (Kool et al., 2019). Attention-based models (Xu et al., 2021) have notably exceeded Recurrent Neural Networks (RNNs) by effectively capturing long-range dependencies. Building on this architecture, Tian et al. (2025) recently proposed a Transformer Model with Parallel Encoders based on the Actor-Critic algorithm to accelerate the calculation of approximate optimal solutions. To manage the high-dimensional action space, researchers employ hierarchical decomposition. Ma et al. (2021) proposed a bi-level framework separating order release and routing. This hierarchical principle is also validated by recent advancements: Deng et al. (2025) applied hierarchical RL to production control under retail uncertainty, while Hu et al. (2025a) utilized multi-agent DRL for multi-echelon inventory optimization. In terms of state representation, Graph Convolutional Networks (GCNs) have become standard for discovering topological relationships. For example, Chen et al. (2024) utilized GCNs for feature prediction to enhance decision accuracy. However, most existing graph-based methods treat all nodes homogeneously. Recent studies suggest that Heterogeneous Graph Attention Networks, which explicitly distinguish between pickup, delivery, and depot nodes, can better capture the asymmetric constraints in PDPs (Li et al., 2021).

Despite these potentials, applying pure RL to DPDPs remains challenging due to sparse rewards and the difficulty of credit assignment over long planning horizons.

2.2.3. Hybrid approaches: Combining learning with search

To mitigate the limitations of standalone RL, specifically sparse rewards and inefficient exploration in complex environments (Guo et al., 2025), recent studies explore hybrid approaches that combine RL's learning capability with the local refinement of heuristics.

One prevalent direction integrates RL with single-point search methods. For instance, Zhang et al. (2023) proposed a model-assisted RL combined with Adaptive Large Neighborhood Search (ALNS) for synchromodal transport. While effective for specific instances, such methods inherently rely on single-trajectory evolution. They refine one solution iteratively, which risks getting trapped in local optima when the reward landscape is deceptive, and incurs high computational costs during large-scale dynamic updates. In contrast, the integration of EA with RL (Evo-RL) offers an alternative to address the exploration-exploitation trade-off (Colas et al., 2018). Unlike single-point hybrids, Evo-RL maintains a diverse population of policies or solutions. The evolutionary operators (selection, crossover, mutation) serve as a global exploration mechanism, uncovering high-quality "stepping stone" solutions even when the RL agent's gradient-based learning stalls due to sparse feedback (Khadka and Tumer, 2018).While traditional RL methods often struggle with exploration in complex, high-dimensional environments, Evo-RL effectively combines RL's efficient local search capabilities with EA's global exploration mechanisms to explore a much wider range of policies, potentially discovering novel and optimal solutions that might remain undiscovered by either method alone (Conti et al., 2018). Several prominent frameworks have validated this synergy. Evolutionary Reinforcement Learning (ERL) (Khadka and Tumer, 2018) combines EA population to generate diverse experiences for DRL training. Deep Evolutionary Reinforcement Learning (DERL) (Gupta et al., 2021) extends this to co-optimize agent morphology and control. However, these frameworks are predominantly designed for continuous control tasks (e.g., robotic locomotion). Their direct application to the DPDP is non-trivial due to the problem's discrete, combinatorial nature and strict operational constraints like LIFO and time windows.

Moreover, the absence of specific adaptation mechanisms for logistics constraints limits the transferability of existing Evo-RL benchmarks. Addressing this gap, our work proposes a tailored Evo-RL framework that adapts the population-based exploration principle to the specific discrete constraints of DPDP, effectively mitigating sparse rewards while ensuring solution feasibility.

3. Problem description

3.1. Problem definition

This section discusses the DPDP in real-world logistics scenarios, specifically in the context of large-scale enterprises such as Huawei. These enterprises must establish a vast transportation network to move billions of products within hundreds of factories each year. However, due to the uncertainties of customers' requirements and corresponding production processes, the majority of delivery orders cannot be fully determined in advance. These delivery orders, which include information such as pickup and delivery

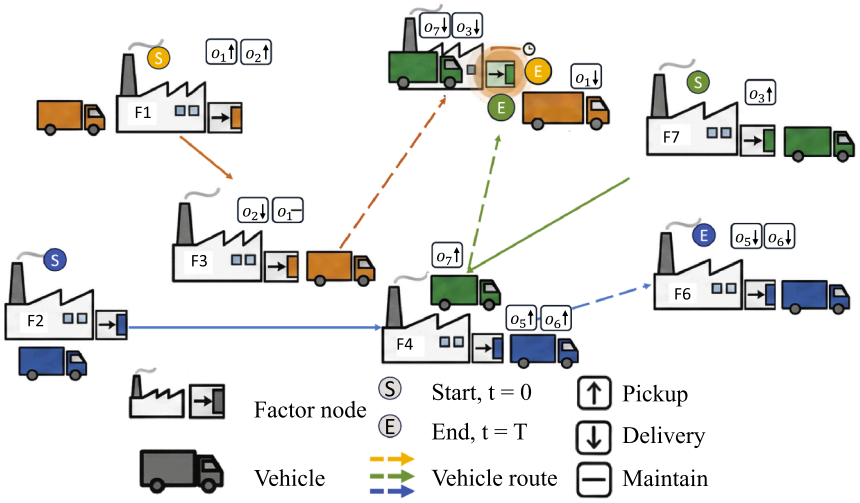


Fig. 1. A snapshot of DPDP, which can be described as: 1. Order o_1 and order o_2 are assigned to the vehicle at factory F_1 (orange truck). The vehicle starts from F_1 and sequentially picks up the goods of both orders o_1 and o_2 (indicated by ↑). The orange truck departs from F_1 after completing pickups and travels to F_3 to deliver order o_2 (o_2 ↓) and maintain at F_3 (o_1–). The orange truck then continues to F_4 to deliver order o_1 (o_1 ↓), but must wait since the green truck is currently occupying the dock at F_4 for delivering order o_3 (o_3 ↓) and picking up order o_7 (o_7 ↓). 2. Order o_3 and order o_7 are assigned to the vehicle at factory F_4 (green truck). The vehicle picks up order o_3 (↑) and delivers order o_7 (↓). After the green truck completes its operations at F_4, it travels to F_7 to pick up order o_3 (o_3 ↑). 3. The blue truck at F_2 (marked with S, Start) is in a maintain state, waiting to begin operations. Order o_5 and order o_6 are assigned to the blue vehicle. Then it goes to F_4 to pick up both orders (o_5 ↑, o_6 ↑) and then delivers order o_5 to factory F_4 (o_5 ↓) and order o_6 to factory F_4 as well (o_6 ↓). 4. The vehicle routes are represented by colored arrows: orange dashed for the orange truck's route, green solid for the green truck's route, and blue dashed for the blue truck's route. Each factory has limited docks, causing potential conflicts when multiple vehicles require the same facility at the same time. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

factories, number of goods, and required turnaround time, are selected randomly based on real-world operational data. To fulfil these orders, a fleet of uniform vehicles is routinely scheduled.

Fig. 1 visualizes the problem as an Open PDP, where vehicle routes are not bound by a return-to-depot requirement. The example highlights a constrained environment where vehicles start from their previous termination points, and factories operate with a single available dock.

In the meantime, this DPDP variant is subject to several constraints, including pickup and delivery, time windows, vehicle capacity, LIFO loading, loading cost, split demand and dock availability. Each factory node has a limited number of docks for loading or unloading cargo, which means that once all the docks are occupied by vehicles, subsequent arriving vehicles must wait in a queue until a dock becomes available. The dock constraints significantly affect the order tardiness of the DPDP, which in turn impacts customer satisfaction with the service. The dispatch order specifies the number of commodities to be picked up, the pickup node, the loading and unloading times, and the required time window. Delivery orders can be dynamically generated by numerous factories throughout the day, with each facility serving as either a pickup or delivery location. Crucially, the dock availability and vehicle movements involve complex stochastic interactions handled by the simulator environment, acting as a "black box" state transition function that is difficult to capture fully with static linear constraints. Therefore, the mathematical model presented below serves as a descriptive formulation to define objectives and operational bounds, rather than a prescriptive MILP model for exact solvers. Our objective is to dispatch every order to a fleet of vehicles, with the aim of reducing the overall tardiness of orders and the average distance travelled by vehicles, minimizing operating costs, and maximizing service quality. To achieve these goals, we consider random occurrence factors such as time windows, varying goods types, vehicle capacity, and pickup and delivery nodes. We make decisions about capacitated vehicles and transshipment locations to convey orders from the origin to the destination while taking into consideration important parameters, including time windows, and loading and unloading rules (such as LIFO). The detailed information is explained in the following section.

3.2. Mathematical model

Since the problem is solved within a simulation environment using learning-based methods, we present a descriptive model that formalizes the optimization goals and constraints.

3.2.1. Inputs and decision variable

The general inputs of the DPDP are:

1. A set of factory nodes $F = \{F_j \mid j = 1, 2, \dots, J\}$, which can be both pickup node and delivery node. Each node is constrained by a certain amount of cargo docks $\mathcal{D}_j = \{D_1, D_2, \dots, D_J\}$ available for loading and unloading as well as the permitted work shifts.The freshly arrived vehicles must wait for their release if all docks are occupied. Additionally, the loading and unloading of goods can only be carried out during specific work shifts at a factory.
2. A road system $R = (F, A)$, which is a complete directed graph. $F = \{F_j \mid j = 1, 2, \dots, J\}$ is the set of factories' nodes. $A = \{(m, n) \mid m, n \in F_j\}$ is the arc set. The transportation time $t^{tt}_{m,n}$ and transportation distance $d^{tt}_{m,n}$ from node m to node n are associated with each arc (m, n).
3. A set of orders $O = \{o_i \mid i = 1, 2, \dots, I\}$ are randomly generated, where each order is $o_i = \{F^{pn}_i, F^{dn}_i, Q_i, t^{gt}_i, t^{cct}_i\}$, including the following information:
   - F^{pn}_i, the pickup factory node.
   - F^{dn}_i, the delivery factory node.
   - $Q_i = \{Q^{standard}_i, Q^{small}_i, Q^{box}_i\}$, the amount of cargoes in the order, where $Q^{standard}_i$ refers to the standard pallet amount, Q^{small}_i refers to the small pallet amount, and Q^{box}_i refers to the amount of boxes. Note that two small pallets are equal to one standard pallet, and two boxes are equal to one small pallet. This setting serves for the split demand constraint.
   - t^{gt}_i, the arrival time of the order.
   - t^{cct}_i, the committed completion time of the order i, which is the sum of dock approaching time $t^{dat}_{k,m}$, dock queuing time $t^{dqt}_{k,m}$, loading time $t^{lt}_{k,m}$, unloading time $t^{ut}_{k,m}$, and travel time $t^{tt}_{k,m,n}$ of the corresponding vehicle v_k.
     - $t^{dat}_{k,m}$, dock approaching time, the period of time a vehicle needs between arriving at the factory and reaching its designated dock, e.g., 30 min, without taking queues into account.
     - $t^{dqt}_{k,m}$, dock queuing time, the amount of time a vehicle waits for an open dock at the factory while every dock there is already full, which is correlated to the occupied docks' least loading or unloading time.
     - $t^{lt}_{k,m} = \omega \times C$ and $t^{ut}_{k,m} = \omega \times C$, loading and unloading time, which is related to the carrying capacity of the vehicle C, while the loading or unloading speed is ω. For example, the loading time $t^{lt}_{k,m} = \omega \times c$ and unloading time $t^{ut}_{k,m} = w \times c$ if there are c standard pallets in the vehicle v_k and $\omega = 180$ s per standard pallet.
     - $t^{tt}_{k,m,n}$, travel time, depending on the chosen road.
4. A fleet of homogeneous vehicles $V = \{v_k, \forall k = 1, 2, \dots, K\}$. Each vehicle has a specific loading capacity C. Vehicles are initialized at specific factory nodes according to the benchmark dataset configuration. While these positions reflect real-world historical states (typically corresponding to the final locations from the previous operational day), from the algorithm's perspective, they constitute a stochastic initial distribution over the factory network.

The primary decision variable at time step t is the aggregate route plan $RP(t) = \{rp_k(t), \forall k = 1, 2, \dots, K\}$ for the entire fleet. Specifically, the route plan for vehicle v_k is defined as the sequence $rp_k(t) = \{r_{k,1}(t), r_{k,2}(t), \dots, r_{k,L_k}(t)\}$, where L_k denotes the number of nodes assigned to vehicle v_k. Furthermore, each element $r_{k,i}(t)$ encapsulates the specific operational details (e.g., pickup or delivery type) for the corresponding visit.

3.2.2. Objective functions

As defined in the problem output, the core decision variables are the route plans $RP = \{rp_k \mid k = 1, \dots, K\}$, where each sequence $rp_k = \{r_{k,1}, r_{k,2}, \dots, r_{k,L_k}\}$ and its length L_k are determined by the optimization algorithm. The objective functions below describe how to evaluate the quality of a generated solution RP.

$$f_1 = \sum_{i=1}^{I} \max\left(0, t^{ft}_i - t^{cct}_i\right), \tag{1}$$

There are two optimization objectives in this DPDP model. The primary aim is to minimize the overall tardiness of all orders. Hence, the total tardiness time is defined as the first objective f_1 in Eq. (1), where t^{ft}_i is the actual arrival time of order o_i at its destination, t^{cct}_i is the committed completion time, and I is the total number of orders. The arrival time t^{ft}_i depends on the sequence of factories visited by the vehicle. Let $idx(i)$ denote the position index in the route rp_k where order o_i is delivered. The arrival time is calculated cumulatively as:

$$t^{ft}_i = t^{start}_k + \sum_{m=1}^{idx(i)} \left(t^{dat}_{k,m} + t^{dqt}_{k,m} + t^{lt}_{k,m} + t^{ut}_{k,m} + t^{tt}_{k,m,m+1}\right), \forall o_i \in O \tag{2}$$

where t^{start}_k is the vehicle's available time, $t^{dat}_{k,m}$ is the approaching time, $t^{dqt}_{k,m}$ is the dynamic queuing time determined for vehicle v_k by the simulator based on dock occupancy at factory F_m , $t^{lt}_{k,m}/t^{ut}_{k,m}$ are loading/unloading time at factory node F_m, and $t^{tt}_{k,m,m+1}$ is the travel time between consecutive nodes in the route rp_k. During the execution, t^{ft}_i is given by the simulator with the route sequence rp_k generated by the decision-making process.

$$f_2 = \frac{1}{K} \sum_{k=1}^{K} \sum_{m=1}^{(L_k-1)} d^{ad}_{r_{k,m}, r_{k,m+1}} \tag{3}$$

The secondary objective is to minimize the average travelling distance of vehicles, represented as f_2 according to Eq. (3), where $d^{ad}_{F_{k,i},F_{k,i+1}}$ is the actual distance from node $F_{k,i}$ to node $F_{k,i+1}$. This objective is crucial in transportation optimization as reducing travel distance can lead to cost savings, fuel efficiency, and overall operational effectiveness.

$$\min f = \Lambda_1 \times f_1 + \Lambda_2 \times f_2. \tag{4}$$

Regarding the problem classification, when multiple objectives are considered simultaneously, such as minimizing waiting time and minimizing travel distance in this case, the problem is typically treated as a multi-objective optimization problem. Specifically, f_1 and f_2 are conflicting objective functions. For instance, in a specific scenario where many orders are generated at different times but share the same pickup and delivery nodes, the objective function f_1 is minimized, while the objective function f_2 is maximized in this situation. Conversely, delaying the previous orders until orders with the same pickup and delivery nodes can be consolidated to be served by a single vehicle will lead to f_1 being maximized and f_2 being minimized. Therefore, it is essential to construct an overall objective function that considers both f_1 and f_2 to achieve a trade-off between them, where improving one objective may lead to a degradation in another. To consolidate these objectives into a single optimization criterion, a simple weighted sum objective is adopted, to follow the same objective used in the competition so that our proposed method can be compared against the winning algorithms. Eq. (4) defines the $\min f$ function that includes both objectives, where Λ_1 and Λ_2 are two constants. Note that the setting of Λ_1 and Λ_2 follows the instructions by the Huawei ICAPS 2021 competition. Improving customer satisfaction and fulfilling orders promptly is crucial for business enterprises in real-world scenarios.

3.2.3. Constraints

Since the problem is solved within a simulation environment using learning-based methods, the following mathematical formulations serve as a descriptive definition of the feasible region and operational rules. The actual enforcement of these constraints, particularly dynamic ones like dock availability and LIFO stacking, is handled by the simulator's validity checks.

The DPDP is restricted by the following constraints:

$$\sum_{k=1}^{K} w_{k,i,F_i^{pn}} \geq 1, \quad \sum_{k=1}^{K} y_{k,i,F_i^{dn}} \geq 1, \quad \forall i \in O \tag{5}$$

$$t_i^{gt} \leq t_i^{ft}, \quad \forall i \in O \tag{6}$$

$$t_{k,F_i^{pn}} < t_{k,F_i^{dn}}, \quad \forall i \in O, \forall k \in V \tag{7}$$

$$0 \leq q^{al}_{k,m} \leq C, \quad \forall k \in V, \forall m \in L_k \tag{8}$$

$$t_{k,F_j^{pn}} > t_{k,F_i^{pn}} \implies t_{k,F_j^{dn}} < t_{k,F_i^{dn}}, \quad \forall i,j \in O_k, \forall k \in V \tag{9}$$

$$\sum_{k=1}^{K} z_{k,m}(t) \leq D_m, \quad \forall m \in F, \forall t > 0 \tag{10}$$

Constraint (5) guarantees that all orders are served effectively. Here, $w_{k,i,F_i^{pn}}$ is a binary indicator equal to 1 if vehicle v_k visits the pickup node of order o_i, and $y_{k,i,F_i^{dn}}$ equals 1 if it visits the delivery node. For standard orders where the cargo quantity Q_i fits within a single vehicle's capacity C, exactly one vehicle is assigned (sum equals 1). However, to satisfy the split demand constraint, if an order's quantity exceeds the vehicle capacity ($Q_i > C$), it is partitioned and served by multiple vehicles, implying that the summation will be greater than 1 to fully cover the total demand. Constraint (6) represents the time window constraint: an order cannot be delivered before it is generated. Note that while there is a committed completion time t_i^{cct}, exceeding it does not render the solution infeasible but incurs a penalty in the objective function f_1. Constraint (7) enforces the precedence requirement: a vehicle must visit the pickup node F_i^{pn} before the matching delivery node F_i^{dn}. Here, $t_{k,Node}$ denotes the time vehicle v_k visits a specific node. Constraint (8) ensures that the cargo load $q^{al}_{k,m}$ of vehicle v_k at any step m along its route never exceeds the maximum capacity C nor drops below zero. Constraint (9) formally describes the LIFO loading discipline. For any two orders o_i and o_j carried by the same vehicle v_k (denoted as set O_k), if order o_j is picked up after order o_i (i.e., $t_{k,F_j^{pn}} > t_{k,F_i^{pn}}$), it must be delivered before order o_i (i.e., $t_{k,F_j^{dn}} < t_{k,F_i^{dn}}$). Constraint (10) represents the dynamic resource constraint handled by the simulator. $z_{k,m}(t)$ is a binary variable indicating whether vehicle v_k is occupying a dock at factory F_m at time t, and D_m is the total number of available docks at factory F_m. It states that the total number of vehicles concurrently occupying docks at factory F_m cannot exceed the available docks D_m.

3.2.4. Uncertain factors

During the practical delivery process, uncertainty factors such as customer preferences and the timing window of orders can make route planning for vehicles challenging (Zhou et al., 2020). In reality, several sources of uncertainty need to be considered, including unpredictable travel times due to traffic jams, bad weather, and other variables, which can cause delays and result in missed pickups and deliveries. Additionally, the travelled distance of vehicles may unexpectedly be longer than expected, leading to service interruptions and changes to the planned routes. These uncertainties can significantly impact the delivery schedule, and as such, effective strategies for managing and mitigating them need to be developed.

In our model, we consolidate the previously mentioned factors, which can be summarized and simplified by uncertainties in time and distance, into two parameters Ψ_i^t and Ψ_i^d that serve as indicators of order satisfaction levels.

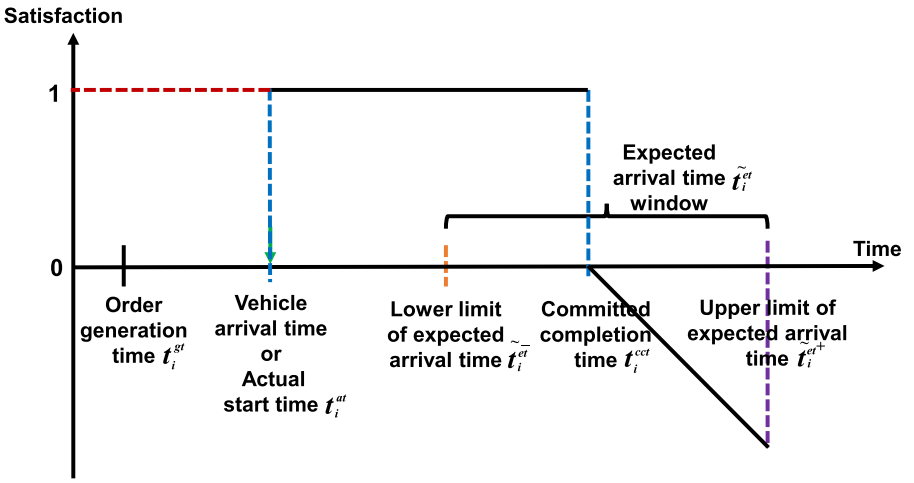


Fig. 2. Order delivery timeline with satisfaction.

Despite the aforementioned uncertainty sources, the primary objective remains to ensure the timely delivery of order o_i within its expected arrival time window, denoted as an interval $[\tilde{t}_i^{et-}, \tilde{t}_i^{et+}]$ with a lower and an upper bound. Therefore, we propose a novel model based on the original model, taking into account the uncertainty associated with the unknown value and probability distribution of the expected order arrival time $\tilde{t}_i^{et}$. Further analysis is conducted to mitigate the impact of uncertainty on the solution.

The actual arrival time t_i^{ft} can be calculated based on Eq. (2). Therefore, when constructing an optimization model, a corresponding adjustment to the aforementioned equation is formulated, depicted as illustrated in Fig. 2. Specifically, the order satisfaction is 1 when the vehicles arrive between t_i^{gt} and t_i^{cct}. Order satisfaction decreases progressively over time with a ratio of 1, i.e., $-\left(\max\left(t_i^{et}, \tilde{t}_i^{et+}\right) - t_i^{cct}\right)$, when the vehicles arrive between t_i^{cct} and $\tilde{t}_i^{et+}$. If the vehicles arrive after $\tilde{t}_i^{et+}$, then the order satisfaction is $-\infty$. It indicates that if a vehicle arrives after a certain time, the order satisfaction is considered to be extremely low, which would be discarded during the optimization process. Thus, the theoretical lower bound does not influence the reward function in the real case.

The order satisfaction about delivery time Ψ_i^t is given as follows:

$$\Psi_i^t = \begin{cases} 1 & t_i^{gt} \le t_i^{ft} \le t_i^{cct} \\ -\left(\max\left(t_i^{et}, \tilde{t}_i^{et+}\right) - t_i^{cct}\right) & t_i^{cct} < t_i^{ft} < \tilde{t}_i^{et+} \\ -\infty & \text{other.} \end{cases} \tag{11}$$

Besides considering the arrival time, minimizing the travelled distance $d_{i,k}^{ed}$ by vehicle v_k in order o_i is the secondary objective of the DPDP problem. The expected travelled distance window $\tilde{d}_{i,k}^{ed}$ can be represented as an interval $[\tilde{d}_{i,k}^{ed-}, \tilde{d}_{i,k}^{ed+}]$ with a lower and upper bound. To account for the uncertainty associated with the unknown value and probability distribution of the expected travelled distance $\tilde{d}_{i,k}^{ed}$, we can develop a robust optimization model based on the original model.

To handle this uncertainty, we can define a satisfaction function Ψ_i^d as Eq. (12) that maps the actual travelled distance $d_{i,k}^{ad}$ to the order satisfaction value. The actual travelled distance $d_{i,k}^{ad}$ can be calculated based on Eq. (3). However, in the presence of uncertainty, a modification to this equation is required to ensure that the actual travelled distance falls within the expected travelled distance window. Specifically, the order satisfaction is within $(0, 1)$ when the vehicle's travelled distance between $\tilde{d}_{i,k}^{ed-}$ and $\tilde{d}_{i,k}^{ed+}$. If the vehicles travelled distance larger than $\tilde{d}_{i,k}^{ed+}$, then the order satisfaction is $-\infty$.

The order satisfaction about delivery distance Ψ_i^d is given as follows:

$$\Psi_i^d = \begin{cases} (0,1) & \tilde{d}_{i,k}^{ed-} \le d_{i,k}^{ad} \le \tilde{d}_{i,k}^{ed+} \\ -\infty & \text{other.} \end{cases} \tag{12}$$

where,$\tilde{d}_{i,k}^{ed-}$ denotes the minimum travelled distance for vehicle v, which just considers the travelled distance for delivering order 1, as $\tilde{d}_{i,k}^{ed-} = d_{F_k^{bn}, F_1^{bn}} + d_{F_1^{bn}, F_1^{fn}} + d_{F_1^{fn}, F_2^{bn}}$, $\tilde{d}_{i,k}^{ed+}$ denotes the maximum travelled distance for vehicle v, which just considers the travelled distance for delivering multi-order $i \in I$, as $\tilde{d}_{i,k}^{ed+} = d_{F_k^{bn}, F_1^{bn}} + \sum_{i=1}^{I} d_{F_i^{bn}, F_i^{fn}} + \sum_{i=1}^{I} d_{F_i^{fn}, F_{i+1}^{bn}}$. Here, F_k^{bn} is the vehicle's start location, and F_i^{bn}/F_i^{fn} correspond to the origin and destination of order i or subsequent orders in the sequence.

By quantifying order satisfaction, we can incorporate it into the reward function. In this case, we define a penalty value for unfulfilled orders as part of the reward function, thereby encouraging the algorithm to prioritize order satisfaction during the planning process, which is illustrated in Section 4.3.3.

4. The proposed Evo-RL approach

In this section, we first provide a brief overview of the proposed algorithm Evo-RL framework before introducing the details of its components.

4.1. General framework

Evolutionary Reinforcement Learning (Evo-RL) is proposed as a holistic hybrid framework that synergizes the sequential decision-making efficacy of DRL with the global search capabilities of GA. Diverging from disjointed modular approaches, Evo-RL establishes a reciprocal feedback loop between the two components. The RL agent functions as a policy-driven initialization mechanism, providing feasible, high-quality solution seeds to constrain and guide the evolutionary search. Conversely, the GA operates as a population-based optimization engine, which not only refines these initial solutions but also enriches the training landscape by generating a diverse spectrum of experience trajectories. This cyclic integration specifically addresses the inherent complexities of the DPDP: the RL component captures latent spatiotemporal dependencies to construct spatially cohesive order batches, establishing a necessary foundation for the subsequent LIFO-compliant sorting, while the GA utilizes its population dynamics to furnish dense, intermediate feedback signals, thereby mitigating the sparse reward problem and stabilizing RL training.

The synergistic architecture of Evo-RL offers distinct advantages over deploying RL or meta-heuristics in isolation. Standard RL typically grapples with the sparse rewards inherent in DPDP, where feedback is often delayed until route completion, and the immense dimensionality of the action space. Evo-RL mitigates these challenges by leveraging the evolutionary population to generate intermediate evaluations, effectively densifying the reward signal. Conversely, traditional meta-heuristics, such as GA or ALNS, often fail to identify feasible starting points under strict LIFO and time window constraints, resulting in computational inefficiency as the algorithm attempts to repair invalid solutions. In our framework, the RL agent captures the latent spatiotemporal dependencies of orders, providing the GA with structurally optimized initial clusters. This effectively prunes the search space, allowing the evolutionary component to focus on refinement rather than feasibility repair.

Fig. 3 illustrates the instantiation of the Evo-RL process. The workflow initiates with the RL agent mapping high-dimensional state observations, comprising order attributes o_i and vehicle states v_k, into a strategic order assignment plan OA. This assignment effectively partitions the global problem into localized sub-problems for each vehicle. To bridge the gap between the RL's high-level allocation and the specific LIFO operational constraints, a heuristic sorting strategy is employed as a translation layer. This layer converts the RL's spatial clustering into temporally valid sequences $o_{k,1}, o_{k,2}, \ldots, o_{k,L_k}$, ensuring that every initial route plan RP strictly adheres to the committed completion times and loading constraints.

These RL-generated sequences serve as the high-quality seeding population for the subsequent evolutionary stage. The GA then executes a broader exploration through constraint-preserving crossover and mutation operators. Crucially, these operators perform dual-level optimization: they not only re-sequence orders within a single route (intra-route optimization) but also exchange orders between different vehicles (inter-route optimization). This allows the GA to rectify suboptimal allocation decisions made by the RL agent while maintaining strict adherence to complex constraints. Finally, the improved route plans generated by the GA are reverse-mapped to the RL's action space (Order Assignments). These refined assignments, along with their fitness evaluations, serve as "pseudo-rewards" to update the RL policy. By exposing the RL agent to these superior variations discovered by the GA, the framework closes the learning loop, enabling the RL agent to progressively generate more evolutionary-friendly and globally optimal initial assignments.

4.2. MDP modeling for DPDP

In this section, we formally define the problem environment and modeling. A Markov Decision Process (MDP) is expressed according to the tuple of $(\mathcal{S}, \mathcal{A}, \mathcal{P}(\cdot), \mathcal{R}_t(\cdot), \gamma)$. Below we detail the specific definitions of these components tailored for the DPDP context.

4.2.1. Environment dynamics

Our framework utilises the high-fidelity DPDP simulator provided by Huawei's ICAPS competition, with a result checking method will be provided to evaluate the performance of the algorithm over the planning horizon (e.g., one day). This black-box environment maintains rigorous operational fidelity by accurately simulating vehicle movements, dock operations, and order fulfillment while strictly enforcing real-world constraints, including LIFO loading, time windows, and capacity limitations. The simulator operates through discrete time steps, at each interval t providing observable state information while internally managing complex system dynamics. The transition probability kernel $\mathcal{P}(s_{t+1} \mid s_t, a_t)$ shows the probability over the next satae s_{t+1} upon performing action a_t from the current state s_t. However, explicitly modeling $\mathcal{P}$ is challenging due to the complex uncertainties. Thus, our trial-and-error learning paradigm becomes essential, as the system's transition dynamics can only be empirically sampled.

4.2.2. State space $\mathcal{S}$

$\mathcal{S}$ denotes the state space or the set of states, which consists of the current status of orders, vehicles, and factories in the DPDP. Specifically, at each time step t, the agent observes $\{O_{i,t}, F_t, V_t\}$:

- The observation of each order at time t is captured by $O_{i,t} = \{F^{fn}_{i,t}, F^{bn}_{i,t}, Q_{i,t}, t^{gt}_{i,t}, t_{m,n,t}, t^{lt}_{k,p,m,t}\}$, where:

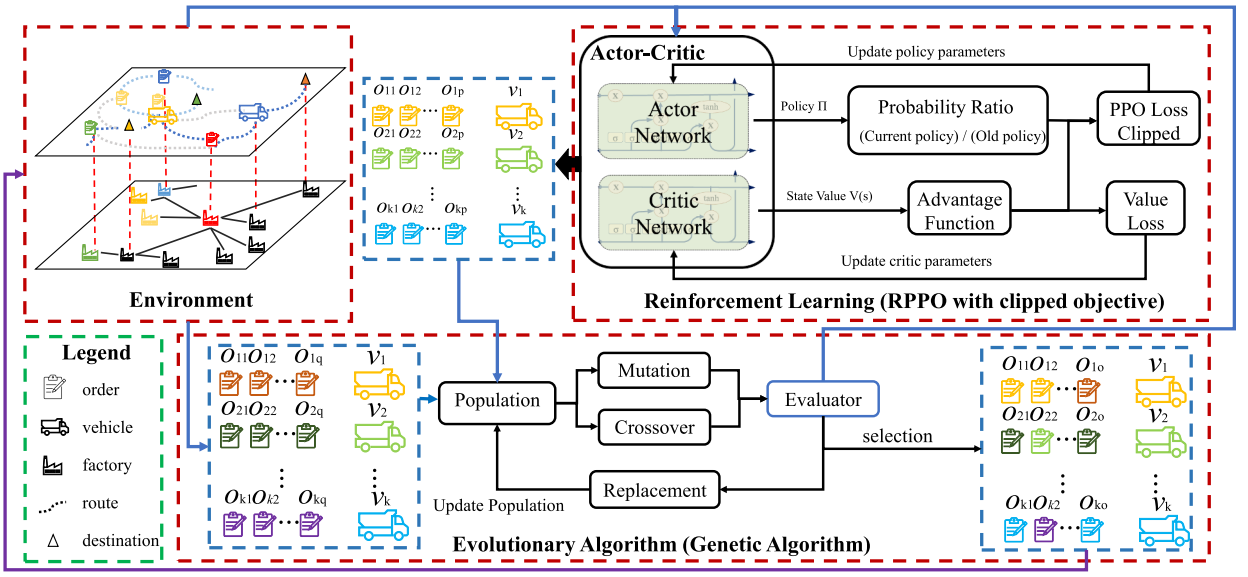


Fig. 3. The proposed Evo-RL framework for solving DPDP.

- $F_{i,t}^{fn}$ and $F_{i,t}^{bn}$ denote the destination and starting factories for order o_i, respectively.
- $Q_{i,t}$ represents the quantity of cargo for order o_i.
- $t_{i,t}^{gt}$ is the generation time for order o_i.
- $t_{m,n,t}$ specifies the transportation time between factories m and n.
- $t_{k,m,t}^{lt}$ indicates the loading time for vehicle v_k for order o_i at node m.

- The observation of the factory status at time t includes $F_t = \{D_{m,t}, N_{m,t}^{\text{tc}}\}$, where:
  - $D_{m,t}$ indicates the number of docks available at factory F_m.
  - $N_{m,t}^{\text{tc}}$ represents the count of vehicles at the factory at time t.
- The observation of each vehicle at time t is described by $V_t = \{F_{k,t}^{cn}, F_{k,t}^{dn}, t_{i,k,t}^{et}, q_{k,t}^{\text{tc}} \mid v_k \in V_t\}$, where:
  - $F_{k,t}^{cn}$ is the current or most recent departure factory of vehicle v_k.
  - $F_{k,t}^{dn}$ is the vehicle's next destination factory.
  - $t_{i,k,t}^{et}$ is the expected arrival time for the current order on vehicle v_k.
  - $q_{k,t}^{\text{tc}}$ is the total cargo load assigned to vehicle v_k on its estimated route.

This rich state representation integrates vital data on orders, factory operations, and vehicle logistics, equipping the RL agent with the necessary information to deduce the optimal vehicle selection policy.

4.2.3. Action space $\mathcal{A}$

In our hybrid framework, we distinguish between the *decision action* generated by the RL agent and the *execution action* required by the simulator.

$\mathcal{A}$ denotes the decision action space for the RL agent, which consists of the assignment of vehicles to orders. At each time step t, the RL agent generates an assignment action $a_t \in \mathcal{A}_t$. An action $a_t = \{v_{k_1}, \ldots, v_{k_n} \mid v_{k_i} \in V\}$ represents the specific vehicle v_{k_i} assigned to order o_i. This action space is discrete and determines the strategic grouping of orders.

Since the simulator requires a concrete sequence of visits (Route Plan, RP) to execute logistics operations, the RL agent's assignment a_t acts as a high-level directive. This directive is operationalized through a deterministic transformation layer (Heuristic Sort + GA refinement) to generate the final executable route plan RP. It is crucial to note that while the RL agent does not directly output the route sequence, the quality of the executable RP is fundamentally bounded by the feasibility of the RL's assignment a_t. If the RL agent groups spatiotemporally incompatible orders, no downstream sorting or evolutionary operator can recover a valid route. Therefore, the MDP is modeled such that the RL agent learns to optimize the *latent feasibility* of order clusters, treating the subsequent routing capability as part of the environment's transition dynamics.

4.2.4. Reward structure $\mathcal{R}$

The primary objective of the DPDP is to maintain efficient transportation operations by minimizing total overtime, waiting times, and travel distances. Formally, the environment provides a sparse feedback signal based on the execution quality of the Route Plan.

We define the fundamental objective function (Environment Reward) as:

$$\mathcal{R}_{env} = -\left[\alpha \cdot \left(\sum_{\forall o_i} \max(0, t_i^{ft} - t_i^{cct})\right) + \beta \cdot \left(\sum_k \text{Distance}(v_k)\right)\right] \tag{13}$$

where α and β are weighting coefficients. However, directly maximizing this sparse reward is difficult for RL agents. Therefore, in the subsequent algorithm section (Section 4.3), we design a dense, augmented reward function to guide the training process.

4.3. Evo-RL optimization algorithm

Based on the MDP formulation, we employ the Evo-RL algorithm to solve the problem. This subsection details the policy optimization via PPO, the evolutionary search via GA, and their collaborative training mechanism.

4.3.1. RL policy optimization and reward shaping

We adopt Proximal Policy Optimization (PPO), a policy-based method, to solve the MDP. To evaluate the policy π, we define the corresponding action-value function (Q-function) $Q^\pi : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ and state-value function $V^\pi : \mathcal{S} \to \mathbb{R}$ as follows:

$$Q^\pi(s,a) = (1-\gamma) \cdot \mathbb{E}\Big[\sum_{t=0}^{\infty} \gamma_t \cdot \mathcal{R}_t(s_t, a_t, s_{t+1}) \mid s_0 = s, a_0 = a, a_t \sim \pi(\cdot \mid s_t), s_{t+1} \sim \mathcal{P}(\cdot \mid s_t, a_t)\Big], \tag{14}$$

$$V^\pi(s) = (1-\gamma) \cdot \mathbb{E}\Big[\sum_{t=0}^{\infty} \gamma_t \cdot \mathcal{R}_t(s_t, a_t, s_{t+1}) \mid s_0 = s, a_t \sim \pi(\cdot \mid s_t), s_{t+1} \sim \mathcal{P}(\cdot \mid s_t, a_t)\Big]. \tag{15}$$

The advantage function $A^\pi(s,a)$ is denoted as

$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s) \tag{16}$$

The policy parameter θ of PPO at the mth iteration is updated by:

$$\theta_{m+1} \longleftarrow argmax_\theta \hat{\mathbb{E}}\left[\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_m}(a_t \mid s_t)} \cdot A_m(s_t, a_t) - \beta \cdot \text{KL}(\pi_\theta(a_t \mid s_t) \| \pi_{\theta_m}(a_t \mid s_t))\right], \tag{17}$$

To address the issue of overfitting, we use the RPPO with a clipped objective. This clipped objective involves calculating a ratio $\aleph_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\theta_m}(a_t \mid s_t)$.

$$\mathfrak{L}_{\theta_k}^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}\left[\sum_{t=0}^{T} [\min(\aleph_t(\theta) A_m(s_t, a_t), \text{clip}(\aleph_t(\theta), 1-\epsilon, 1+\epsilon) A_m(s_t, a_t))]\right]. \tag{18}$$

Augmented Reward Function Design. To overcome the sparsity of the environmental reward and enforce feasibility constraints, we design a composite reward function for the RL agent. This function augments the ground-truth environment reward with intrinsic signals and penalty terms.

The total reward r_t used for updating the policy is defined as:

$$r_t = r_{intrinsic} + r_{feasibility} + r_{collaborative} \tag{19}$$

1. Intrinsic Efficiency Reward: Encourages the agent to reduce delays and distances at each step:

$$r_{intrinsic} = \lambda_1 \left(\sum_{\forall o_i \in O_t} \max\left(0, t_i^{ft} - t_i^{cct}\right)\right) + \lambda_2 \left(\frac{1}{K_t} \sum_{\forall v_k \in V_t} d_{i,k}^{ad}\right) \tag{20}$$

2. Feasibility Punishment: To enforce the strict constraints (LIFO and Time Windows) discussed in Section 4.2, we apply a penalty $r_{feasibility}$ whenever the RL's assignment leads to an infeasible route:

$$r_{feasibility} = \lambda_4 \Sigma W_t = \lambda_4 (\widehat{\Upsilon_{i\,t}^t} + \widehat{\Upsilon_{i\,t}^d}) \tag{21}$$

where $\widehat{\Upsilon_i^t}$ and $\widehat{\Upsilon_i^d}$ penalize violations of time windows and travel limits, respectively:

$$\widehat{\Upsilon_i^t} = \begin{cases} 1 & t_i^{gt} \le t_i^{ft} \le t_i^{cct} \\ 10 \cdot \left(\max\left(t_i^{et}, \tilde{t_i^{et}}^+\right) - t_i^{cct}\right) & t_i^{cct} < t_i^{ft} < \tilde{t_i^{et}}^+ \\ 1000 & \text{otherwise.} \end{cases} \tag{22}$$

$$\widehat{\Upsilon_i^d} = \begin{cases} 1 & \tilde{d_k^{ed}}^- \le d_k^{ad} \le \tilde{d_k^{ed}}^+ \\ 1000 & \text{otherwise.} \end{cases} \tag{23}$$

3. Collaborative Feedback (Pseudo-Reward): Leveraging the Evo-RL framework, we incorporate the relative performance of the GA population into the reward:

$$r_{collaborative} = \lambda_3 \delta G_t = \lambda_3 (\varpi_t - \varrho_t) \tag{24}$$

where ϖ_ℓ and ϱ_ℓ denote the scores of the RL solution and the best GA-evolved solution, respectively. This term encourages the RL agent to generate assignments that serve as better "seeds" for the evolutionary process.

Finally, the final reward function r_f reflects the ultimate goal at the end of the episode:

$$r_f = \Lambda_1\left(\sum_{\forall o_i \in O} \max\left(0, t_i^{ft} - t_i^{cct}\right)\right) + \Lambda_2\left(\frac{1}{K}\sum_{k=1}^{K}\sum_{i=1}^{L_k-1} d_{F_{k,i},F_{k,i+1}}\right), \tag{25}$$

where, Λ_1 and Λ_2 are the updated optimal coefficients.

4.3.2. GA for route planning

In this work, we select GA over trajectory-based meta-heuristics for the routing component to specifically address the issue of reward sparsity in RL training. The effectiveness of the Evo-RL framework relies on feeding diverse, high-quality solution data back into the RL agent to serve as a replay buffer. A population-based GA inherently generates multiple diverse candidate solutions (individuals) in parallel at each generation. These candidates are assessed by an efficient evaluator to produce "pseudo-rewards", providing the RL agent with a dense and rich gradient of feedback signals that a single-trajectory method like ALNS cannot easily offer. GA utilize genetic operators as part of its search and optimization process. In general, there are two genetic operators, the crossover operator, and the mutation operator. The purpose of this operator is to combine two input solutions within a given state space, resulting in a new route plan. The output obtained by the RL algorithm is reformulated as a chromosome for each vehicle v_k. The chromosome consists of a solution list of orders for each vehicle: $H_k = o_{k,1}, o_{k,2}, \ldots, o_{k,l}$. Besides that, the initial population of solutions is crucial for the exploration of the solution space and the convergence speed of the algorithm. One of the initial solutions can be a solution obtained from an RL algorithm, which potentially represents a high-quality starting point due to the RL's ability to learn and adapt to the problem environment. The remaining solutions can be created through random generation, ensuring a variety of genetic material in the initial population. This diversity is essential as it allows the GA to explore various regions of the solution space, preventing premature convergence on suboptimal solutions and maintaining genetic diversity throughout the evolution process. For each vehicle, a sorted order sequence abiding by the LIFO principle can be obtained: $o_{k,1}, o_{k,2}, \ldots, o_{k,L_k}$.

Crossover operators employ tournament selection to select sequence fragments from both parent chromosomes, exchanging allocated orders between different chromosomes and altering the order assignments for each vehicle.

$$P(H_k \times G_k \to \mathcal{W}_k) = \mathcal{P}_c \frac{1}{\mathcal{L}-1}\sum_{\varsigma} \varkappa(H_k, G_k, \mathcal{W}_k, \varsigma), \tag{26}$$

where, $\mathcal{P}_c$ denotes the probability of crossover operator, H_k denotes the solution string generated by RL as parent string, G_k denotes the randomly generated solutions string as parent strings, $\mathcal{W}_k$ denotes the potential descendant solution string, $\varsigma \in [1, \ldots, \mathcal{L}-1]$ denotes the randomly selected crossover location and is assumed to be uniformly distributed, $\varkappa(\cdot)$ denotes the crossover function takes values from 0 to 1, where 1 indicates that the solution string $\mathcal{W}_k$ is producing by crossing H_k and G_k at location ς.

Mutation operators are applied to modify the order sequences within individual vehicles, such as swapping the execution order of two orders within a vehicle's sequence. Mutation operates on each individual independently by perturbing the solution string in a probabilistic manner. The flipping of the jth bit in the ith individual solution is a stochastically independent event that occurs with a probability of $\mathcal{P}_m$, where $\mathcal{P}_m$ is a value between 0 and 1. This probabilistic process allows for random changes in the genetic information of individuals, and the occurrence of flipping a specific bit is not influenced by the flipping of other bits or the state of other individuals. The probability that solutions string H_k resembles solutions string $H_k^{'}$ after mutation can be aggregated to:

$$P\{H_k \to H_k^{'}\} = \mathcal{P}_m^{M(H_k,H_k^{'})}(1-\mathcal{P}_m)^{\mathcal{L}-M(H_k,H_k^{'})}, \tag{27}$$

where, $M(H_k, H_k^{'})$ denotes the Hamming distance of the pair $(H_k, H_k^{'})$, represents the number of solutions that need to be changed through mutation to transform H_k into $H_k^{'}$, $\mathcal{L}$ denotes the length of solutions string.

The formulation of the conditional probability of constructing a solution $\mathcal{W}_k$ through crossover and mutation, given a current population described by U, as:

$$P(\mathcal{W}_{k+1} \mid U) = \mathcal{P}_c \cdot \sum_{H_k^{'} \in U}\sum_{H_k \in U} P(H_k^{'} \mid U)P(H_k \mid U) \cdot \frac{1}{\mathcal{L}-1}\sum_{\varsigma}\varkappa(H_k, G_k, \mathcal{W}_k, \varsigma) + (1-\mathcal{P}_c)\cdot P(\mathcal{W}_k \mid U), \tag{28}$$

where, $P(H_k^{'} \mid U)$ is given based on Eq. (27).

Parents are selected from the population with the highest score by the evaluator. The evaluator assesses all order assignment plan $OA = \{(o_i, v_k) \mid o_i \in O, v_k \in V\}$, including the initial solution generated by the RL algorithm, and chooses the best-estimated solution for execution. The fitness evaluation is performed by the evaluator, which calculates the fitness value E as follows:

$$E = \lambda_1 * (t_i^{et} - t_i^{cct}) + \lambda_2 * (t_i^{et} - t_i^{fct}) + \lambda_3 * t_k^{ot} + \lambda_4 * t_{k,m}^{dqt}. \tag{29}$$

In the fitness evaluation equation, several components are considered to optimize route planning. The current overtime for unexecuted orders that will exceed the deadline after execution is calculated as $(t_i^{et} - t_i^{cct})$, representing the additional time required to complete unexecuted orders beyond their deadlines. The inefficiency in the execution of completed orders is quantified as $(t_i^{et} - t_i^{fct})$, where t_e^i is the estimated completion time, and t_i^{fct} is the theoretical fastest completion time. t_k^{ot} is the overtime of orders from the previous period, accounting for any delays in order execution that have carried over from earlier time periods. Lastly, the queuing

time for a vehicle v_k waiting for an open dock at factory m is represented by $t_{k,m}^{dqt}$. This term captures the time that vehicles spend waiting for available docks at factories, which can impact the overall efficiency of the transportation system.

With Fitness Proportional Selection (FPS), originally introduced by Holland (Holland, 1992), the probability of selecting a candidate solution $\eta \in U$ is proportional to its fitness relative to the total fitness of the current population. Let $U = \{\eta_1, \eta_2, \ldots, \eta_{|U|}\}$ denote the current population set, where each individual element represents a complete route plan for the entire vehicle fleet. Then, the conditional selection probability of η is given by

$$P(\eta \mid U) = \frac{E_\eta}{\sum_{i=1}^{|U|} E_i}, \tag{30}$$

where E_i denotes the fitness value of individual i.

Following the Eq. (30), as the number of populations increases in a GA, the time required for each iteration of the GA also tends to increase. This can result in longer optimization times and a greater amount of time needed to find a comparable solution.

To obtain comparable solutions within a relatively short period, we update the crossover and mutation probabilities based on the fitness value. The specific updating process is as follows:

$$\mathcal{P}_c = \begin{cases} \mathcal{P}_{cmax} - \dfrac{\mathcal{P}_{cmax} - \mathcal{P}_{cmin}}{\Phi} \cdot \phi & \text{for} \quad E' \geq E_{avg} \\ \mathcal{P}_{cmax} & \text{for} \quad E' < E_{avg} \end{cases} \tag{31}$$

$$\mathcal{P}_m = \begin{cases} \mathcal{P}_{mmax} - \dfrac{\mathcal{P}_{mmax} - \mathcal{P}_{mmin}}{\Phi} \cdot \phi & \text{for} \quad E' \geq E_{avg} \\ \mathcal{P}_{mmax} & \text{for} \quad E' < E_{avg} \end{cases} \tag{32}$$

where, $\mathcal{P}_{cmax}$ denotes the maximum crossover probability, $\mathcal{P}_{cmin}$ denotes the minimum crossover probability, Φ denotes the maximum number of iterations, $\phi \in \Phi$ denotes the current iteration count, $\mathcal{P}_{mmax}$ denotes the maximum mutation probability, $\mathcal{P}_{mmin}$ denotes the minimum mutation probability, E' denotes the greater fitness value in two individuals who want to perform variation operations, E_{avg} denotes the mean fitness value of the population.

By adjusting the crossover and mutation probabilities in Eqs. (31) and (32), we can control the diversity and exploration level of new solutions to achieve better fitness values. This approach helps to ensure that the generated solution strings are comparable to RL solutions, which can leverage the search capabilities of GA to assist RL in exploring a broader solution space.

4.3.3. Integrating RL and GA

A collaborative training and evaluation process is conducted to integrate RL and GA for the proposed framework (see Algorithm 1 for code level details). In the overall Evo-RL framework, the integration of RL and GA goes beyond a simple sequential execution; they form a symbiotic feedback loop. We initialize the optimization process with the RL agent to perform vehicle selection. This choice is critical because the RL agent, trained on accumulated environmental feedback, can learn the complex, non-linear dependencies between order attributes and vehicle states, which are not easily captured by a simple distance-based heuristic. By providing a high-quality initial allocation, the RL agent allows the GA to start its search in a promising region of the solution space, preventing the GA from wasting generations on infeasible or low-quality areas. The GA's evaluator assesses the solutions within the population, including those from the RL algorithm. In the evaluator, we choose to relax some constraints to reduce the complexity and time of simulation, to estimate an approximated reward. By evaluating these solutions, the GA estimates the next state and provides rewards for the RL agent. The population's states, actions, and rewards are collectively fed into the RL algorithm. The RL agent can efficiently update its policy to optimize vehicle selection by employing batch training. This integrated approach allows the proposed framework to learn and adapt to the dynamic transportation environment, effectively balancing exploration and exploitation in the search for high-quality solutions.

In other words, RL mechanisms estimate the optimal solution by utilizing information from previous populations in GA to accelerate computation toward the DPDP, when a change is detected in the environment. The best solution Ξ of Evo-RL feed into the environment can be formulated as:

$$\Xi = \text{argmax}\{H_k, \eta\}, \tag{33}$$

where, H_k denotes the RL solution and η denotes the GA solution.

In the context of GA, the population excluding the best candidate, which is selected for execution in the environment, is evaluated using pseudo rewards. These pseudo rewards, approximated of the actual rewards, are generated by the evaluator and are based on the GA's action populations. By integrating pseudo rewards with actual rewards, the learning gradient of the policy within RL can be significantly enhanced. The RL policy update at each iteration m is formulated as:

$$\theta_{m+1} \longleftarrow argmax_\theta \hat{\mathbb{E}}\left[\frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_m}(a_t \mid s_t)} \cdot (\lambda_c \cdot A_m(s_t, a_t)) - \beta \cdot \text{KL}(\pi_\theta(a_t \mid s_t) \| \pi_{\theta_m}(a_t \mid s_t))\right], \tag{34}$$

where, λ_c denotes the confidence coefficient, which is used to scale the contribution of estimated advantages to the policy update, mitigating undue reliance on evaluations that may lack precision.

By iteratively refining vehicle selection and route planning, the integrated proposed framework can learn and adapt to the dynamic transportation environment. The RL model offers the ability to learn from historical data in a one-step dynamic setting, effectively

Algorithm 1 The Proposed Evo-RL Approach.

Input: Number of episodes L, steps per episode T, crossover probability $\mathcal{P}_c$, mutation probability $\mathcal{P}_m$, maximum number of GA iterations Φ, population size $|U|$
Initialize: Policy π, policy parameter θ, a replay buffer $\mathbb{B}$
for l = 1, 2,…,L **do**
  for t = 1, 2,…, T **do**
    for order o_i in O_t **do**
      Collect $\{O_{i,t}, F_t, V_t\}$ as current observation
      Compute the order assignment based on current policy π_{θ_t}
    end for
    Collect a set of order assignment OA and convert it into route plan H_k
    Initialize population U_0 with H_k and G_k
    Evaluate fitness of individuals in U_0 with the evaluator
    for $\phi = 1, 2, \ldots, \Phi$ **do**
      Select parents from U_ϕ that maximize E in Eq. (29).
      Use Partial-Mapped Crossover to generate M_c children with probability $\mathcal{P}_c$ to generate offspring
      Apply mutation with probability $\mathcal{P}_m$ to offspring
      Evaluate fitness using the evaluator with E in Eq. (29).
      Update U_ϕ using a replacement strategy
    end for
    Choose the best solution Ξ in Eq. (33) and execute Ξ in the environment
    Observe next state $\{O_{t+1}, F_{t+1}, V_{t+1}\}$ and calculate the reward r_t
    Use evaluator to get E as the estimated reward for the other selected populations
    Push operation transition $\{\{O_t, F_t, V_t\}, \Xi, \{O_{t+1}, F_{t+1}, V_{t+1}\}, r_t, \eta, E\}$ into $\mathbb{B}$
  end for
  Sample a batch of transitions from $\mathbb{B}$
  Calculate the state value V^π in (15)
  Calculate the advantage A^π in (16)
  Update π_θ using Eq. (17)
  for i = 1, 2, …, $|U| - 1$ **do**
    Calculate V_i^π and A_i^π with η_i and E_i
    Update π_θ using Eq. (34)
  end for
end for

handling uncertainty within the problem and improving the quality of initial solutions throughout the training process. This, in turn, reduces the iteration time for the GA population. The population information from the GA serves as a source of exploration for the RL algorithm, accelerating the convergence speed during the training process. This collaborative approach enables the algorithm to determine high-quality solutions that minimize overtime, reduce vehicle waiting times, and maintain efficient transportation operations.

5. Experimental results

5.1. Dataset and environment simulator

To evaluate the effectiveness of our proposed Evo-RL algorithm on the DPDP, we employ the Huawei ICAPS 2021 dataset[1]. This dataset provides a comprehensive representation of real-world logistics operations, including delivery orders, the dynamics of order transactions, vehicle constraints, and the specifics of 153 factories across 64 benchmark cases. The information on delivery orders includes pickup and delivery factories, quantity of goods, and time requirements. These orders occur randomly and are periodically serviced by fleets of homogeneous vehicles, which presents a realistic and challenging environment for testing our algorithm. We employ the high-fidelity simulator with built-in result verification provided by Huawei's competition platform. This simulation environment features a closed-loop interaction mechanism that faithfully replicates real-world logistics operations while strictly enforcing all operational constraints. The standardized simulation framework not only ensures experimental validity but also facilitates direct performance comparison with other benchmark algorithms.

For our experiments, we define a total of eight distinct scenarios as follows: (1) 5 vehicles with 50 orders, (2) 5 vehicles with 100 orders, (3) 20 vehicles with 300 orders, (4) 20 vehicles with 500 orders, (5) 50 vehicles with 1000 orders, (6) 50 vehicles with 2000 orders, (7) 100 vehicles with 3000 orders, and (8) 100 vehicles with 4000 orders. These scenarios are intended to evaluate

[1] https://competition.huaweicloud.com/information/1000041411/introduction.

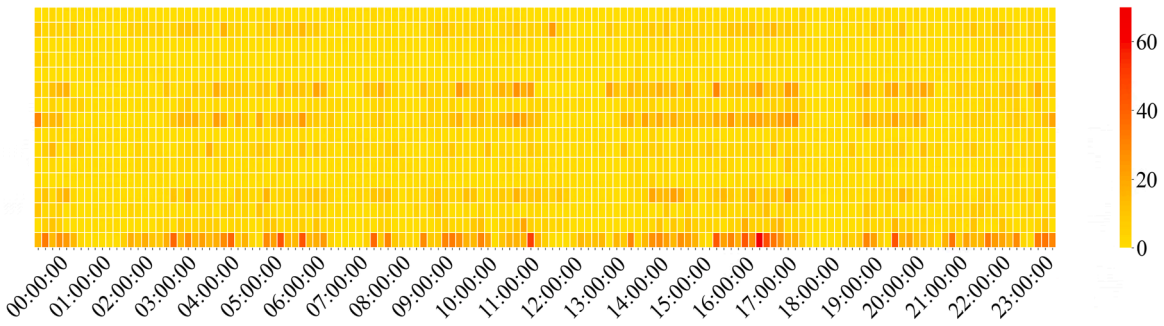


Fig. 4. Spatial-temporal distribution of delivery demand of a one-day large case with a total of 153 factories. Each day is split into 144 time intervals of equal duration. The darker a grid is, the stronger the delivery demand of the grid is.

the scalability and adaptability of the Evo-RL algorithm under varying degrees of demand and fleet size. To ensure fair comparison, all experiments adopt the standard configuration parameters from the Huawei ICAPS 2021 competition baseline: (1) in objective function (4), $\Lambda_1 = 10000, \Lambda_2 = 1$ (derived from estimates within the Huawei logistics distribution system); (2) a maximum of 6 docks per factory; (3) vehicle maximum load capacity Q restricted to 15; and (4) a constrained runtime limit of 10 min per instance. Fig. 4 visually depicts the order distribution for a single day in the most extensive case involving 4000 orders. This illustration highlights the complexity and the spatiotemporal challenges that our Evo-RL algorithm is capable of addressing in large-scale logistics operations.

5.2. Results and discussion

To rigorously assess the performance of our Evo-RL framework, we have chosen a diverse array of comparative methods used to solve the DPDP as comparisons, which include:

- The standalone RL component, which corresponds to our proposed framework without the evolutionary (Evo) module, utilizes the Recurrent PPO (RPPO) algorithm (Pu et al., 2023). This RL approach effectively exploits the temporal structure of the problem to enhance policy learning.
- Hierarchical heuristic (Hier) (Du et al., 2023): Current SOTA approach for the same problem. We reproduce the hierarchical optimization approach that alternates between three order-dispatch rules (urgent, hitch-hike, packing-bags) and LIFO-compliant block-based local search.
- The top three open-sourced winning heuristic algorithms from the ICAPS 2021 competition, known for their effectiveness in DPDP:
  - Variable Neighborhood Search (VNS), a robust metaheuristic that systematically explores the solution space by varying the neighbourhood structures, allowing for the escape from local optima. This method is denoted as **W1** in comparative results.
  - A domain-specific heuristic, combines manual design principles with industry insights, offering a solution finely tuned to the nuances of the DPDP. This method is denoted as **W2** in comparative results.
  - Local Search with Clever Neighborhood, a method that enhances traditional local search strategies with a novel neighbourhood design to efficiently navigate complex solution spaces. This method is denoted as **W3** in comparative results.
- ADP (Jiang et al., 2023): We implement an ADP framework that employs neural network-based value function approximation to address the large state-action space inherent in our DPDP problem. The method utilizes post-decision states, obtained by interacting with the environment, and Bellman equation updates to optimize long-term reward expectations. To ensure fair comparison with our Evo-RL approach, we maintain identical network architectures with the RPPO baseline, while deliberately omitting the evolutionary refinement phase to evaluate ADP's core performance in isolation.
- GA, configured to adhere to the iteration time constraint, reflects a classic evolutionary computation strategy adapted to the DPDP context.
- Baseline solution provided by Huawei, a straightforward, exhaustive search approach provided by Huawei to quantify the efficiency gains of more sophisticated methods. This baseline employs an insertion originally designed for static PDP (Savelsbergh and Sol, 1998), adapting it to the dynamic context by re-optimizing upon each event. Specifically, upon the release of each new order, all incomplete pickup and delivery nodes are integrated into the current routes of available vehicles to yield a feasible solution, effectively treating each dynamic event as a new static subproblem.

Our evaluation aims to establish the competitive edge and superior performance of the proposed Evo-RL algorithm in handling dynamic pickup and delivery scenarios. Table 2 presents the average scores obtained by the six algorithms for various test cases, reflecting the efficiency and effectiveness of each approach in minimizing the cost function defined by the problem. Table 3 displays the average overtime duration, indicating the algorithms' ability to meet the time constraints imposed by the dynamic environment. In the table, the best algorithm result for each type of case is marked in bold, and the second best is underlined. In scenarios involving 5 vehicles and 50 orders, the proposed Evo-RL method exhibits a 11% improvement over the best-performing heuristic (Hierarchical). This performance gain is further accentuated with an increase in problem size; for instance, in cases with 5 vehicles and 100 orders, our method outperforms the next best heuristic (W2) by 32%. The GA encountered scalability limitations, failing to produce results within

Table 2
Average score for different cases.

Score / Methods / Cases		Evo-RL (Ours)	Hier	W1	W2	W3	RL(RPPO) (Pu et al., 2023)	ADP (Jiang et al., 2023)	GA (Wang et al., 2022)	Base-line(Hao et al., 2022)
vehicle	order number									
5	50	**1240 (11%)**	1386	1531	2951	4832	86,452	78354	18,477	157,938
5	100	**85345 (32%)**	188,521	314,320	126002	252,455	1,352,789	1493226	330,911	2,425,982
20	300	**11772 (40%)**	34,516	36,548	25,340	19775	988,792	871413	Timeout	5,310,690
20	500	**108615 (24%)**	143208	176,654	227,829	319,981	3,543,256	3721077	Timeout	56,935,332
50	1000	**98633 (29%)**	138574	763,803	471,765	253,264	2,273,564	2795713	Timeout	68,619,555
50	2000	**215547 (77%)**	960604	3,248,765	5,613,854	3,145,405	36,935,332	42110302	Timeout	826,096,163
100	3000	**1355420 (3%)**	1,931,681	1,985,003	1390308	4,994,369	126,548,783	114724580	Timeout	1,209,884,573
100	4000	**1764734 (20%)**	2213428	2,291,704	3,765,994	5,210,054	419,781,319	542670412	Timeout	1,834,457,967

Table 3
Average timeout for different cases.

Time / Methods / Cases		Evo-RL (Ours)	Hier	W1	W2	W3	RL(RPPO) (Pu et al., 2023)	ADP (Jiang et al., 2023)	GA (Wang et al., 2022)	Base-line(Hao et al., 2022)
vehicle	order number									
5	50	**432**	466	479	995	1688	31,021	28149	5877	52,841
5	100	**30654**	78,919	113,061	72538	90,765	486,907	518464	101,598	873,244
20	300	**4160**	8464	13,053	9048	7018	355,868	313506	Timeout	1,911,775
20	500	**35981**	60144	63,513	81,927	115,093	1,275,488	1336372	Timeout	20,496,633
50	1000	**35424**	48233	274,892	169,754	91,097	811,092	894702	Timeout	24,702,950
50	2000	**77540**	325650	1,169,445	2,020,901	1,132,250	13,296,644	15062245	Timeout	297,394,517
100	3000	**487872**	688,757	714,525	500414	1,797,858	45,557,475	42091336	Timeout	435,558,372
100	4000	**635279**	802209	824936	1,355,662	1,875,514	151,121,179	195415162	Timeout	660,404,755

the time constraints for larger cases, denoted by Timeout. In stark contrast, the proposed Evo-RL method demonstrates exceptional scalability, successfully computing solutions across all tested scenarios. Notably, with 50 vehicles handling 2000 orders, the proposed method achieves a 77% better score than the top-performing heuristic. This substantial enhancement in operational efficiency is further corroborated in the largest evaluated scenario, with 100 vehicles and 4000 orders, where the proposed method outpaces the leading heuristic by 20%. Compared to the baseline method, the proposed framework offers an extraordinary 86.5% improvement in score, indicative of a pronounced increase in operational efficiency. The heuristic methods, while viable for smaller and less complex cases, exhibit diminishing returns as they grapple with the dynamic intricacies and uncertainties of larger-scale problems. The proposed Evo-RL algorithm, on the other hand, consistently demonstrates remarkable robustness and scalability, particularly in more demanding and extensive instances. This notable performance is attributed to the synergistic integration of evolutionary strategies and reinforcement learning within the Evo-RL framework, empowering the algorithm to consistently deliver high-quality solutions amidst the complexities of dynamic transportation environments. Moreover, the traditional RL method's performance deteriorates rapidly with increasing problem scale. For example, in the most extensive scenario of 100 vehicles and 4000 orders, the proposed Evo-RL method outperforms the traditional RL approach by a staggering 99.6%. This pronounced disparity underscores the efficacy of the evolutionary components within the Evo-RL framework, enabling it to adeptly optimize within the context of large-scale and volatile problem landscapes.

The box plots in Fig. 5 provide a comparative visualization of the distribution of overtime durations and corresponding vehicle travel distances for the challenging scenario of managing 4000 orders with 100 vehicles. The box plots demonstrate that the proposed method achieves significantly lower average overtime durations and maintains a tighter distribution, indicative of its consistent performance and efficient order allocation strategy. Notably, its interquartile range (IQR) for both metrics is markedly narrower than competing methods, underscoring robustness against operational variability. While the RL, similarly to the ADP method, display a wider range and higher median in overtime durations, suggesting a less consistent and potentially inefficient order allocation approach. Their overlapping distributions further imply limited performance differentiation between RL and ADP in this task domain. Similarly, the heuristic methods (W1-W3) show inconsistent performance regarding vehicle travel distances and overtime durations, with W2 exhibiting pronounced outliers in travel distance, a sign of unstable routing decisions under high-order volume. The shortened travel distances achieved by the proposed method, along with a decrease in variance, further confirm the effectiveness of the proposed approach.

The distribution of order timeout durations across different time intervals for different-scale cases is presented in Fig. 6. The colour intensity within these figures corresponds to the length of overtime durations, with darker shades representing longer periods of delay. In the small case (Fig. 6a), which involves 5 vehicles and 100 orders, our method shows limited instances of overtime, confined to just three distinct time intervals. This is in contrast with the other methods, where a greater frequency and extent

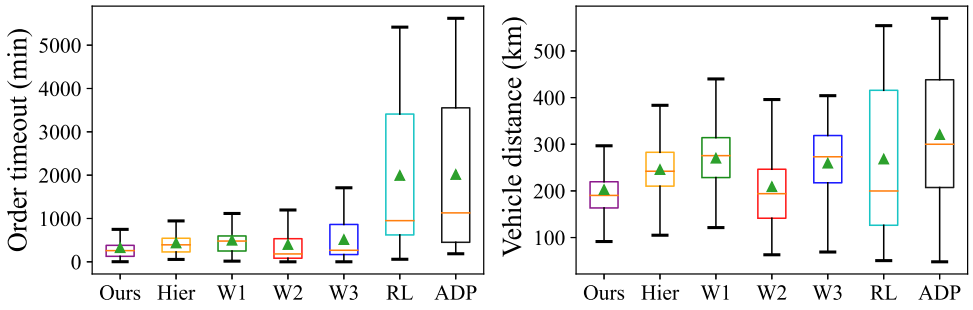
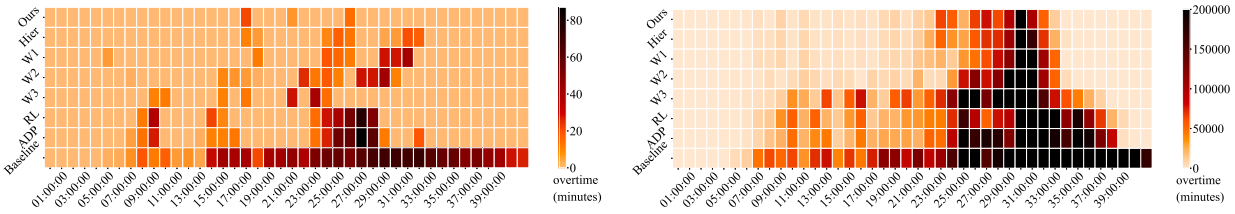


Fig. 5. Distribution of quantitative data for vehicles' travelling distance and order timeout.



(a) Sum of each hour's over time for different methods in 5 vehicles 100 orders case.

(b) Sum of each hour's over time for different methods in 100 vehicles 4000 orders case

Fig. 6. Sum of each hour's over time for different methods in different cases.

of overtime periods are observed. Notably, the ADP algorithm exhibits a similar overtime pattern to PPO, with marginally higher intensity during peak intervals (e.g., 10:00-12:00), reflecting shared limitations in these ways. In the large-scale case (Fig. 6b), which involves 100 vehicles and 4000 orders, a notable increase in overtime is observed after 20:00. This surge is attributed to the bulk of orders being placed between 15:00 and 17:00, as shown in Fig. 4, and the subsequent concentration of these orders in a limited number of factories. Constraints such as a finite number of docks and considerable loading times contribute to increased waiting durations. Both learning-based methods (RL and ADP) display nearly identical overtime accumulation trends, with ADP showing slightly delayed mitigation post-22:00, likely due to its slower convergence in high-dimensional action spaces. Furthermore, once an order is underway, the inflexibility to adjust task objectives may result in suboptimal allocation, exacerbating the backlog and leading to prolonged overtime durations. Our proposed Evo-RL method implements a more strategic approach, effectively mitigating the cumulative overtime, as indicated by the lighter color shades and earlier completion times depicted in the figure. These results highlight the superior adaptability and performance of our method, particularly in handling complex, large-scale logistics scenarios. To deepen the analysis, we examine the execution of orders during the peak period of 24:00 to 25:00 for the case of 100 vehicles and 4000 orders.

The order execution status during the peak period is presented in Fig. 7, which provides a visualization of the geographical locations of each factory (blue dots), the status of ongoing orders (solid grey lines), and the timeout durations for expired orders (coloured lines). The colour intensity of the coloured lines corresponds to the length of the delay for the expired orders. For the proposed algorithm, despite the challenging peak period, the accumulated order timeout duration is relatively shorter compared to the other heuristic methods. In contrast, the other heuristic methods exhibit prolonged uncompleted orders, often as a result of ineffective handling of split orders and a failure to process the separate parts in a timely fashion. This leads to extended cumulative timeouts, where orders remain unfinished for a longer duration. This comparison can be attributed to the algorithm's ability to process previously unfinished orders promptly, ensuring a more efficient flow of order execution.

The performance of the proposed method has been shown in Fig. 8. In particular, Fig. 8a displays the busy status of a 5-vehicle fleet across various time slices when tasked with handling an ascending number of orders, specifically comparing scenarios with 50, 75, and 100 orders. The graph demonstrates a strong correlation between vehicle activity levels, order generation intervals, and the number of orders within those intervals. This relationship suggests that the proposed Evo-RL algorithm is effectively managing fleet operations to accommodate the incoming demand. Moreover, the effectiveness of our algorithm is not only evident in its ability to handle increased order volumes but also in its capacity to optimize scheduling on a per-vehicle basis. Fig. 8b provides a detailed view of the ongoing order numbers for each vehicle in a scenario with 100 orders distributed among five vehicles. The figure reflects the dynamic allocation of orders to individual vehicles over time, indicating the algorithm's proficiency in load balancing and time management under a higher operational load.

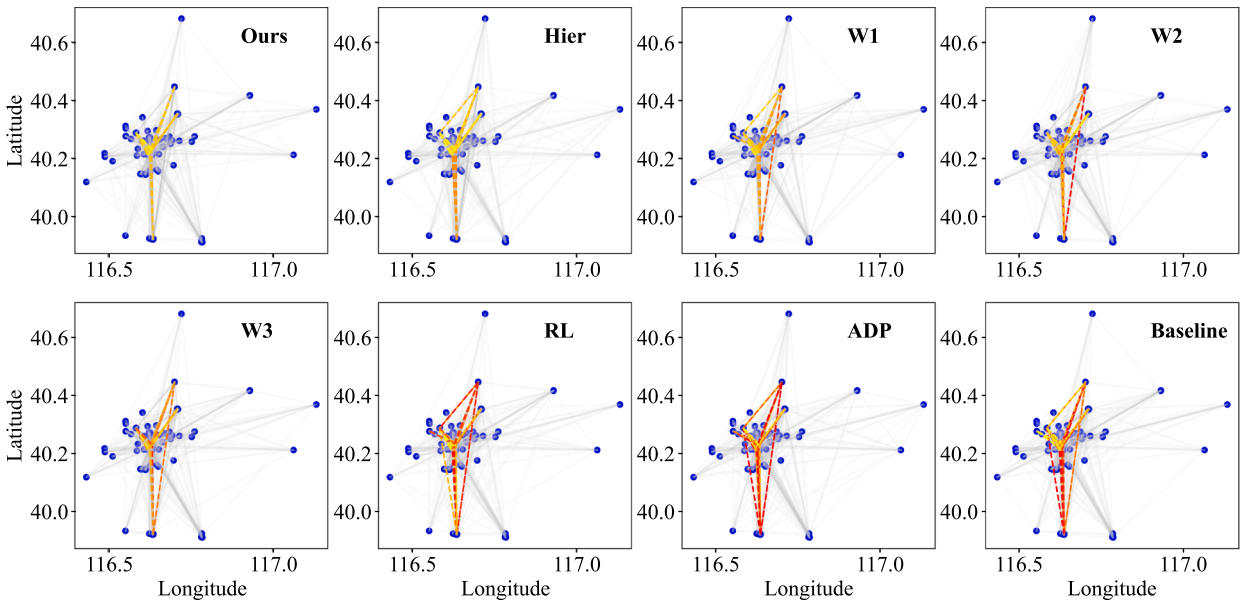
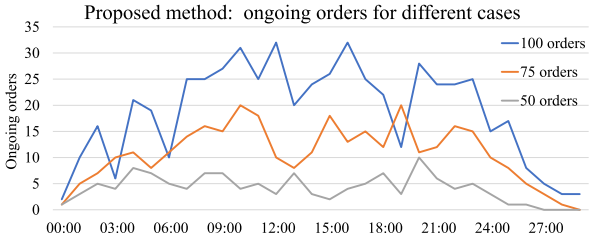
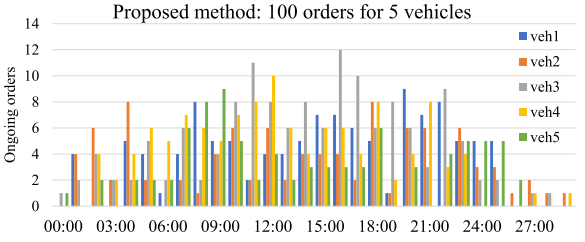


Fig. 7. Visualization of order execution status during the peak period of 24:00 to 25:00 for the case with 100 vehicles and 4000 orders, comparing the timeout durations across different algorithms. The proposed Evo-RL algorithm shows shorter accumulated timeouts, indicating its effective management even during busy periods.



(a) Busy status of a 5-vehicle fleet over time for scenarios with increasing order quantities (50, 75, and 100 orders), showcasing the algorithm's ability to maintain high vehicle utilization across different demand levels.



(b) Dynamic order allocation to each vehicle over time in the 5-vehicle and 100-order scenario, demonstrating the algorithm's load balancing and time management capabilities.

Fig. 8. Performance of the proposed Evo-RL on different scenarios.

5.3. Convergence analysis

This section analyzes the convergence behavior of the baseline RL algorithm and the proposed Evo-RL framework across environments with varying fleet sizes. For fairness, we applied identical training budgets and hyperparameters across methods, varied only by the presence of evolutionary operations in Evo-RL. Fig. 9 summarises the results for four fleet-size scenarios, where the vertical axis shows the objective-function score (lower is better) and the horizontal axis shows the number of training episodes. Solid curves are the mean scores across five independent random seeds; shaded bands depict 95% confidence intervals obtained by local-resampling perturbation of the single-seed training log.

To quantify convergence, we define the convergence episode as the earliest episode at which the smoothed objective (e.g., using a moving average window of 50 episodes) enters and remains within a $\pm\delta$ band of its final plateau value (we set δ based on the variance observed in the last 10% of training episodes). Under this criterion, Evo-RL consistently converges faster and to lower objective values than standard RL:

- 5 vehicles: Evo-RL converges at approximately 80 episodes versus 200 episodes for RL.
- 20 vehicles: Evo-RL converges at approximately 900 episodes versus 1600 episodes for RL.
- 50 vehicles: Evo-RL converges at approximately 2400 episodes versus 7000 episodes for RL.
- 100 vehicles: Evo-RL converges at approximately 10,000 episodes versus 18,000 episodes for RL.

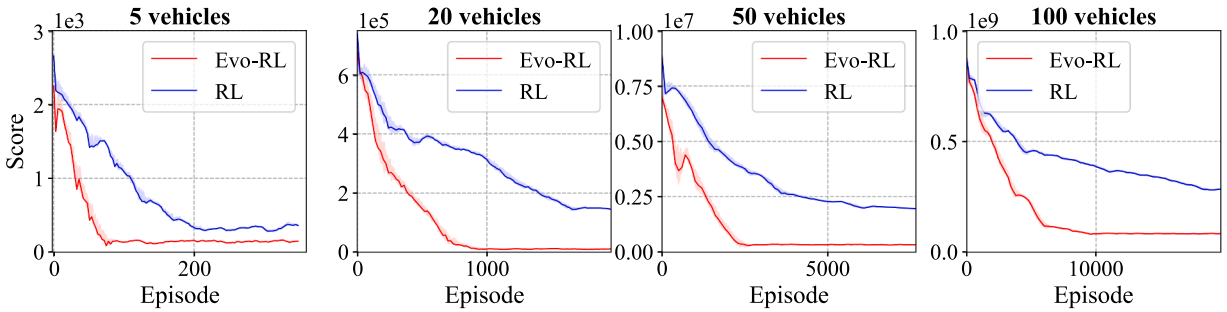


Fig. 9. Learning curves for the RL baseline and the proposed Evo-RL method under four vehicle-density scenarios. Solid lines represent average performance over five independent random seeds; shaded bands indicate 95% confidence intervals estimated by local-resampling perturbation of the single-seed log.

Table 4
Average score for different cases.

Cases		Score		Time	
Vehicle	Order number	Evo-RL	VNS-RL	Evo-RL	VNS-RL
5	50	**1240**	8766	**432**	2714
5	100	**85345**	92,217	**30654**	45,379
20	300	**11772**	166,533	**4160**	10,253
20	500	**108615**	2,554,127	**35981**	210,398
50	1000	**98633**	3,118,648	**35424**	302,011
50	2000	**215547**	48,752,053	**77540**	8,846,421
100	3000	**1355420**	92,684,012	**487872**	6,583,717
100	4000	**1764734**	25,642,176	**635279**	20,384,584

These results suggest two key advantages of Evo-RL: (i) accelerated convergence driven by population-level exploration and solution refinement via GA operators, and (ii) improved asymptotic performance, reflected by lower objective scores at convergence. The evolutionary component broadens policy search and mitigates premature convergence typical in RL-only training, while the RL component exploits high-quality candidates identified by GA, yielding superior final solutions.

5.4. Population-based benefits: Comparison with VNS-RL

This section evaluates the viability and advantages of incorporating a population-based evolutionary component for solving the DPDP. To this end, we introduce VNS-RL, a simplified variant of the proposed Evo-RL framework that omits the evolutionary population. Instead, VNS-RL couples a Variable Neighborhood Search (VNS) procedure with RL to accelerate convergence toward feasible solutions.

For a controlled ablation, VNS replaces the GA component while preserving the overall architecture: RL is responsible for order allocation, and the search module performs route planning based on RL-generated assignments. The VNS is tailored to the dynamic DPDP setting by supporting real-time insertion of new orders and enforcing LIFO loading constraints. Unlike traditional VNS variants designed for static instances or continuous domains, our implementation employs a dynamic neighborhood structure and a flexible solution representation that enables rapid route updates under tight LIFO compliance (i.e., the last package loaded is the first delivered). Upon the arrival of a new order, the VNS incrementally adjusts the incumbent solution rather than restarting the search, using specialized neighborhoods, such as order insertion, route-segment swapping, and vehicle reassignment, to quickly produce feasible updates. These neighborhoods explicitly validate and adjust pickup-delivery sequences to maintain stack-based loading order. The RL agent further guides neighborhood selection based on its learned policy, focusing VNS exploration on promising regions of the search space. This tailored synergy allows VNS-RL to adapt to the dynamic problem state while respecting DPDP constraints.

We compare Evo-RL and VNS-RL to quantify the contribution of the population-based component to overall performance and scalability. Table 4 summarizes average objective scores and total overtime across scenarios with varying fleet sizes and order volumes. In the smallest case (5 vehicles, 50 orders), Evo-RL attains an objective score that is 85.9% lower than VNS-RL, indicating a substantial improvement in solution quality; total overtime is reduced by 84.1%, reflecting higher efficiency. As complexity increases, the performance gap widens: with 20 vehicles and 500 orders, Evo-RL achieves a 95.7% lower score and 82.9% less overtime than VNS-RL, demonstrating effective scaling. In the largest scenario (100 vehicles, 4000 orders), Evo-RL's score is 93.1% lower, and total overtime is 96.9% less than VNS-RL. These empirical results consistently highlight the benefits of population-based exploration and refinement: Evo-RL converges faster and reaches superior objective values, particularly in large-scale, high-uncertainty instances.

Table 5
Testing time of different methods for different cases.

Second / Methods / Orders	Evo-RL (Ours)	Hier	W1	W2	W3	RL	VNSRL	ADP	GA	Baseline
100	**1.6**	4.2	174	28	78	1.7	1.6	2.3	522	104
500	**1.8**	8.7	208	33	141	1.8	1.9	2.6	738	152
2000	**2.2**	35	231	37	206	3.1	2.7	4.6	1851	216
4000	**2.5**	268	412	54	384	3.8	3.5	4.7	2043	295

5.5. Computational costs

Table 5 reports the average wall-clock time per decision epoch (10-min time slice) required by each method during testing. All experiments were conducted on a workstation equipped with an Intel Xeon Gold 5416S processor, 128GB RAM, and an NVIDIA A5000 GPU with 24GB of memory. Classic meta-heuristics (GA, W1-W3) and the model-based Hierarchical heuristic scale super-linearly: their run-time grows by two to three orders of magnitude when the number of orders increases from 100 to 4000. This behaviour is consistent with the theoretical expectation that local-improvement and neighbourhood-exploration routines visit an exponentially growing number of LIFO-feasible sequences as problem size increases (Du et al., 2023). RL-based agents (RL, ADP, VNSRL and the proposed Evo-RL) exhibit almost flat scaling: inference simply performs one forward pass through the trained network, followed by a constant-time action-selection step. Consequently, even for the largest 100-order instances our Evo-RL policy decides vehicle-order assignments in 2.5 s on average, which is two orders of magnitude faster than the best competing heuristic (W2: 54 s) and three orders of magnitude faster than the Hierarchical method (268 s).

Moreover, because Evo-RL produces higher-quality decisions, the cumulative number of unfinished orders at any step remains smaller; hence the average number of requests that must be handled per decision cycle is reduced, further shortening the wall-clock time reported in the table. The evolutionary wrapper embedded in Evo-RL adds only a negligible overhead compared with the standalone RL baseline, confirming that the genetic search is executed offline during training and does not burden real-time operations. The above observations corroborate the key advantage of the proposed Evo-RL framework: it inherits the sub-second inference speed of neural policies while still enjoying the high solution quality delivered by evolution-enhanced training. Therefore, in time-critical logistics platforms where new orders arrive every few seconds, Evo-RL offers a scalable and deployable alternative to traditional heuristics whose latency quickly violates the dispatching deadline.

6. Conclusion

We presented Evo-RL, a collaborative framework designed to address the high-dimensional and dynamic nature of the DPDP under strict LIFO constraints. By integrating a population-based GA with Deep RL, we established a closed-loop system where the GA serves not only as a local optimizer but also as a generator of dense feedback signals. This synergy effectively mitigates the reward sparsity often encountered in dynamic logistics environments, allowing the RL agent to learn robust policies despite the complexity of side-loading constraints.

Our experimental results on the ICAPS 2021 benchmark demonstrate that this hybrid approach significantly outperforms both standalone heuristics and deep RL baselines in terms of convergence speed and solution quality. Crucially, the incorporation of a stochastic uncertainty model into the intrinsic reward mechanism enabled the agent to anticipate potential delays, resulting in routing schedules that are resilient to real-world variability.

On a broader level, this work suggests that for combinatorial problems constrained by rigid physical protocols (such as side-loading vehicles), population-guided RL offers a distinct advantage over single-trajectory search methods. By decoupling the strategic planning (handled by RL) from the sequence refinement (handled by GA), the framework avoids the local optima that trap traditional solvers. Future research will extend this methodology to heterogeneous fleets and multi-objective optimization, specifically aiming to balance operational efficiency with carbon emission reduction in large-scale supply chains. Future research will explore the generalization of this collaborative architecture to other constrained combinatorial problems, such as electric vehicle routing with non-linear charging functions.

CRediT authorship contribution statement

Xiaoying Yang: Writing – original draft, Visualization, Validation, Investigation, Formal analysis, Data curation; **Yiran Li:** Writing – original draft, Visualization, Validation, Investigation, Formal analysis; **Tianxiang Cui:** Writing – review & editing, Writing – original draft, Supervision, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Conceptualization; **Ruibin Bai:** Writing – review & editing, Supervision, Funding acquisition.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This Project is Supported by the Ningbo Science and Technology Bureau, China (Ningbo Commonweal Programme, Project ID 2025S114; Ningbo Young Scientific and Technological Innovation Leading Talent Scheme, Project ID 2025QL055). This Project is also Supported by the Shanxi Provincial Department of Education Higher Education Institution Technology Innovation Program (Project ID 2025L152), and by the Ningbo Science and Technology Bureau, China (Project ID 2023Z237, 2025Z197).

References

Agussurja, L., Cheng, S.-F., Lau, H.C., 2019. A state aggregation approach for stochastic multiperiod last-mile ride-sharing problems. Transp. Sci. 53 (1), 148–166.
Al-Kanj, L., Nascimento, J., Powell, W.B., 2020. Approximate dynamic programming for planning a ride-hailing system using autonomous fleets of electric vehicles. Eur. J. Oper. Res. 284 (3), 1088–1106.
Bai, R., Chen, X., Chen, Z.-L., Cui, T., Gong, S., He, W., Jiang, X., Jin, H., Jin, J., Kendall, G., et al., 2023. Analytics and machine learning in vehicle routing research. Int. J. Prod. Res. 61 (1), 4–30.
Bombelli, A., Fazi, S., 2022. The ground handler dock capacitated pickup and delivery problem with time windows: a collaborative framework for air cargo operations. Transp. Res. E Logist. Transp. Rev. 159, 102603.
Cai, J., Zhu, Q., Lin, Q., Ma, L., Li, J., Ming, Z., 2023. A survey of dynamic pickup and delivery problems. Neurocomputing , 126631.
Cai, J., Zhu, Q., Lin, Q., Ming, Z., Tan, K.C., 2024. Decomposition-based multiobjective evolutionary optimization with tabu search for dynamic pickup and delivery problems. IEEE Trans. Intell. Transp. Syst. 25 (10), 14830–14843.
Chakrabarty, A., Jha, D.K., Buzzard, G.T., Wang, Y., Vamvoudakis, K.G., 2020. Safe approximate dynamic programming via kernelized lipschitz estimation. IEEE Trans. Neural Netw. Learn. Syst. 32 (1), 405–419.
Chen, J.-F., Wang, L., Liang, Y., Yu, Y., Feng, J., Zhao, J., Ding, X., 2024. Order dispatching via GNN-based optimization algorithm for on-demand food delivery. IEEE Trans. Intell. Transp. Syst. 25 (10), 13147–13162.
Chen, X., Ulmer, M.W., Thomas, B.W., 2022. Deep q-learning for same-day delivery with vehicles and drones. Eur. J. Oper. Res. 298 (3), 939–952.
Colas, C., Sigaud, O., Oudeyer, P.-Y., 2018. GEP-PG: decoupling exploration and exploitation in deep reinforcement learning algorithms. In: International Conference on Machine Learning. PMLR, pp. 1039–1048.
Conti, E., Madhavan, V., Petroski Such, F., Lehman, J., Stanley, K., Clune, J., 2018. Improving exploration in evolution strategies for deep reinforcement learning via a population of novelty-seeking agents. In: Advances in neural information processing systems 31.
Cui, T., Ding, S., Jin, H., Zhang, Y., 2023. Portfolio constructions in cryptocurrency market: a CVar-based deep reinforcement learning approach. Econ. Model. 119, 106078.
Cui, T., Du, N., Yang, X., Ding, S., 2024. Multi-period portfolio optimization using a deep reinforcement learning hyper-heuristic approach. Technol. Forecast. Soc. Change 198, 122944.
Deng, Y., Chow, A. H.F., Yan, Y., Su, Z., Zhou, Z., Kuo, Y.-H., 2025. Hierarchical production control and distribution planning under retail uncertainty with reinforcement learning. Int. J. Prod. Res. , 63, (12), 4504–4522.
Du, J., Zhang, Z., Wang, X., Lau, H.C., 2023. A hierarchical optimization approach for dynamic pickup and delivery problem with LIFO constraints. Transp. Res. E Logist. Transp. Rev. 175, 103131.
Duan, J., He, Z., Yen, G.G., 2021. Robust multiobjective optimization for vehicle routing problem with time windows. IEEE Trans. Cybern. 52 (8), 8300–8314.
Dulac-Arnold, G., Levine, N., Mankowitz, D.J., Li, J., Paduraru, C., Gowal, S., Hester, T., 2021. Challenges of real-world reinforcement learning: definitions, benchmarks and analysis. Mach. Learn. 110 (9), 2419–2468.
Feng, L., Huang, Y., Zhou, L., Zhong, J., Gupta, A., Tang, K., Tan, K.C., 2020. Explicit evolutionary multitasking for combinatorial optimization: a case study on capacitated vehicle routing problem. IEEE Trans. Cybern. 51 (6), 3143–3156.
Fruit, R., Pirotta, M., Lazaric, A., Ortner, R., 2018. Efficient bias-span-constrained exploration-exploitation in reinforcement learning. In: International Conference on Machine Learning. PMLR, pp. 1578–1586.
Ge, X., Yin, Q., Moktadir, M.A., Ren, J., 2025. Dynamic routing optimization of electric vehicles for retailers based on consumer behavior prediction. Transp. Res. E Logist. Transp. Rev. 204, 104417. https://doi.org/10.1016/j.tre.2025.104417
Gendreau, M., Guertin, F., Potvin, J.-Y., Séguin, R., 2006. Neighborhood search heuristics for a dynamic vehicle dispatching problem with pick-ups and deliveries. Transp. Res. C Emerg. Technol. 14 (3), 157–174.
Ghiani, G., Manni, A., Manni, E., 2022. A scalable anticipatory policy for the dynamic pickup and delivery problem. Comput. Oper. Res. 147, 105943.
Ghiani, G., Manni, E., Quaranta, A., Triki, C., 2009. Anticipatory algorithms for same-day courier dispatching. Transp. Res. E Logist. Transp. Rev. 45 (1), 96–106.
Guo, T., Mei, Y., Zhang, M., Zhao, H., Cai, K., Du, W., 2025. Learning-aided neighborhood search for vehicle routing problems. IEEE Trans. Pattern Anal. Mach. Intell. 47 (7), 5930–5944. https://doi.org/10.1109/TPAMI.2025.3554669
Gupta, A., Savarese, S., Ganguli, S., Fei-Fei, L., 2021. Embodied intelligence via learning and evolution. Nat. Commun. 12 (1), 5721.
Hao, J., Lu, J., Li, X., Tong, X., Xiang, X., Yuan, M., Zhuo, H.H., 2022. Introduction to the dynamic pickup and delivery problem benchmark–ICAPS 2021 competition. arXiv preprint arXiv:2202.01256
Holland, J.H., 1992. Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence. MIT press.
Hu, J., Xia, L., Huang, T., Wu, H., 2025a. A multi-agent deep reinforcement learning approach for multi-echelon inventory optimization and its application to the beer game. Transp. Res. E Logist. Transp. Rev. 203, 104367.
Hu, T., Yang, X., Jia, F., Xu, H., Yang, K., Cui, T., 2025b. An enhanced multi-objective evolutionary algorithm for adaptive-formation multi-UAV task allocation with load-dependent constraints. In: 2025 IEEE Congress on Evolutionary Computation (CEC). IEEE, pp. 1–8.
Jeong, J., Moon, I., 2024. Dynamic pickup and delivery problem for autonomous delivery robots in an airport terminal. Comput. Ind. Eng. 196, 110476.
Jia, Y.-H., Mei, Y., Zhang, M., 2021. A bilevel ant colony optimization algorithm for capacitated electric vehicle routing problem. IEEE Trans. Cybern. 52 (10), 10855–10868.
Jiang, K., Cao, Y., Zhou, H., Wu, J., Zhang, Z., Liu, Z., 2023. Adaptive dynamic programming for multi-driver order dispatching at large-scale. IEEE Trans. Cogn. Commun. Netw. 10 (2), 607–621.
Jin, J., Cui, T., Bai, R., Qu, R., 2023. Container port truck dispatching optimization using real2sim based deep reinforcement learning. Eur. J. Oper. Res. 315, 161–175.
Khadka, S., Tumer, K., 2018. Evolution-guided policy gradient in reinforcement learning. In: Advances in Neural Information Processing Systems. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2018/file/85fc37b18c57097425b52fc7afbb6969-Paper.pdf.
Kool, W., van Hoof, H., Welling, M., 2019. Attention, learn to solve routing problems! In: International Conference on Learning Representations. https://openreview.net/forum?id=ByxBFsRqYm.

Li, J., Xin, L., Cao, Z., Lim, A., Song, W., Zhang, J., 2021. Heterogeneous attentions for solving pickup and delivery problem via deep reinforcement learning. IEEE Trans. Intell. Transp. Syst. 23 (3), 2306–2315.
Li, M., Cai, K., Zhao, P., 2025. Optimizing same-day delivery with vehicles and drones: a hierarchical deep reinforcement learning approach. Transp. Res. E Logist. Transp. Rev. 193, 103878.
Liu, S., Tang, K., Yao, X., 2021. Memetic search for vehicle routing with simultaneous pickup-delivery and time windows. Swarm Evol. Comput. 66, 100927.
Ma, Y., Hao, X., Hao, J., Lu, J., Liu, X., Xialiang, T., Yuan, M., Li, Z., Tang, J., Meng, Z., 2021. A hierarchical reinforcement learning based optimization framework for large-scale dynamic pickup and delivery problems. Adv. Neural Inf. Process. Syst. 34, 23609–23620.
Mes, M. R.K., Rivera, A.P., 2017. Approximate dynamic programming by practical examples. In: Markov Decision Processes in Practice. Springer, pp. 63–101.
Mitrović-Minić, S., Krishnamurti, R., Laporte, G., 2004. Double-horizon based heuristics for the dynamic pickup and delivery problem with time windows. Transp. Res. B Methodol. 38 (8), 669–685.
Mousavi, K., Bodur, M., Cevik, M., Roorda, M.J., 2024. Approximate dynamic programming for pickup and delivery problem with crowd-shipping. Transp. Res. B Methodol. 187, 103027.
Muñoz-Carpintero, D., Sáez, D., Cortés, C.E., Núñez, A., 2015. A methodology based on evolutionary algorithms to solve a dynamic pickup and delivery problem under a hybrid predictive control approach. Transp. Sci. 49 (2), 239–253.
Omidvar, M.N., Li, X., Yao, X., 2021. A review of population-based metaheuristics for large-scale black-box global optimization-part i. IEEE Trans. Evol. Comput. 26 (5), 802–822.
Osaba, E., Yang, X.-S., Fister, , I.Jr, Del Ser, J., Lopez-Garcia, P., Vazquez-Pardavila, A.J., 2019. A discrete and improved bat algorithm for solving a medical goods distribution problem with pharmacological waste collection. Swarm Evol. Comput. 44, 273–286.
Peppel, M., Ringbeck, J., Spinler, S., 2022. How will last-mile delivery be shaped in 2040? A delphi-based scenario study. Technol. Forecast. Soc. Change 177, 121493.
Psaraftis, H.N., Wen, M., Kontovas, C.A., 2016. Dynamic vehicle routing problems: three decades and counting. Networks 67 (1), 3–31.
Pu, Z., Wang, H., Liu, Z., Yi, J., Wu, S., 2023. Attention enhanced reinforcement learning for multi agent cooperation. IEEE Trans. Neural Netw. Learn. Syst. 34 (11), 8235–8249. https://doi.org/10.1109/TNNLS.2022.3146858
Pureza, V., Laporte, G., 2008. Waiting and buffering strategies for the dynamic pickup and delivery problem with time windows. INFOR: Inf. Syst. Oper. Res. 46 (3), 165–175.
Sáez, D., Cortés, C.E., Núñez, A., 2008. Hybrid adaptive predictive control for the multi-vehicle dynamic pick-up and delivery problem based on genetic algorithms and fuzzy clustering. Comput. Oper. Res. 35 (11), 3412–3438.
Savelsbergh, M., Sol, M., 1998. Drive: dynamic routing of independent vehicles. Oper. Res. 46 (4), 474–490.
Swihart, M.R., Papastavrou, J.D., 1999. A stochastic and dynamic model for the single-vehicle pick-up and delivery problem. Eur. J. Oper. Res. 114 (3), 447–464.
Thomas, D., Eric, H., Anja, H., Bernd, H., Christoph, K., Richa, S., Christoph, W., 2020. The future of the last-mile ecosystem. World Econ. Forum, Geneva, Switzerland 1–28. https://www.weforum.org/publications/the-future-of-the-last-mile-ecosystem/
Tian, R., Sun, Z., Chang, L., Wu, J., Lu, X., 2025. Rapid solution for flexible pickup and delivery services problem based on improved actor-critic deep reinforcement learning. IEEE Trans. Intell. Transp. Syst. 26 (6), 7640–7654.
Ulmer, M.W., Thomas, B.W., Campbell, A.M., Woyak, N., 2021. The restaurant meal delivery problem: dynamic pickup and delivery with deadlines and random ready times. Transp. Sci. 55 (1), 75–100.
Wang, Y., Ran, L., Guan, X., Fan, J., Sun, Y., Wang, H., 2022. Collaborative multicenter vehicle routing problem with time windows and mixed deliveries and pickups. Expert Syst. Appl. 197, 116690.
Wolfinger, D., Salazar-González, J.-J., 2021. The pickup and delivery problem with split loads and transshipments: a branch-and-cut solution approach. Eur. J. Oper. Res. 289 (2), 470–484.
Xiao, J., Zhang, T., Du, J., Zhang, X., 2019. An evolutionary multiobjective route grouping-based heuristic algorithm for large-scale capacitated vehicle routing problems. IEEE Trans. Cybern. 51 (8), 4173–4186.
Xu, X., Wei, Z., 2023. Dynamic pickup and delivery problem with transshipments and LIFO constraints. Comput. Ind. Eng. 175, 108835. https://doi.org/10.1016/j.cie.2022.108835
Xu, Y., Fang, M., Chen, L., Xu, G., Du, Y., Zhang, C., 2021. Reinforcement learning with multiple relational attention for solving vehicle routing problems. IEEE Trans. Cybern. 52 (10), 11107–11120.
Xue, N., Bai, R., Jin, H., Cui, T., 2025. An evolutionary method with shift pattern learning for real-world multi-skilled personnel scheduling with flexible shifts. Swarm Evol. Comput. 99, 102160.
Yang, K., Zhang, Z., Zhao, J., Liang, Z., 2025. A neighborhood-based matheuristic for the vehicle routing problem with delivery options. Transp. Res. E Logist. Transp. Rev. 203, 104366.
Zhang, Y., Bai, R., Qu, R., Tu, C., Jin, J., 2022. A deep reinforcement learning based hyper-heuristic for combinatorial optimisation with uncertainties. Eur. J. Oper. Res. 300 (2), 418–427.
Zhang, Y., Negenborn, R.R., Atasoy, B., 2023. Synchromodal freight transport re-planning under service time uncertainty: an online model-assisted reinforcement learning. Transp. Res. C Emerg. Technol. 156, 104355.
Zhou, M., Liu, X.-B., Chen, Y.-W., Qian, X.-F., Yang, J.-B., Wu, J., 2020. Assignment of attribute weights with belief distributions for MADM under uncertainties. Knowledge-Based Syst. 189, 105110.